MAHATMA EDUCATION SOCIETY'S

PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Empowered Autonomous)

NEW PANVEL

PROJECT REPORT ON

"DEVASCEND: DEVELOPER GROWTH PREDICTOR"

IN PARTIAL FULFILLMENT OF
M.Sc. DATA ANALYTICS
SEMESTER I 2026-27

PROJECT GUIDE
Prof. Shivapradeep Muthupandi

SUBMITTED BY: **Razi Ahmed Mumtaz**
ROLL NO: **2857**

Mahatma Education Society's

PILLAI COLLEGE OF ARTS,COMMERCE & SCIENCE

(Empowered Autonomous)

Re-accredited "A" Grade by NAAC (3rd Cycle)

Project Completion Certificate

THIS IS TO CERTIFY THAT

RAZI AHMED MUMTAZ

of **M.Sc. Data Analytics Part - I** has completed the project titled **"DevAscend: Developer Growth Predictor"** of subject **Foundation of Data Science** under our guidance and supervision during the academic year 2026-27 in the department of Computer Science.

Project Guide	Course Coordinator	Head of the Department

Pillai College of Arts, Commerce & Science (Autonomous) New Panvel

INDEX

Sr. No.	Topic
1	Cover Page
2	Project Completion Certificate
3	Introduction (About the Project)
4	Tools and Techniques Used
5	Code and Output Screenshots
6	Conclusion

3. INTRODUCTION

3.1 About the project

DevAscend predicts a software developer's current level, market value and future growth, and recommends what to learn next. It is trained on **153,645 professional developers** from the **Stack Overflow Developer Surveys 2021–2025**. All **360,519 raw responses** are stored in **Apache Cassandra**, and the training data is read back from Cassandra.

- **Level classifier (XGBoost)** predicts Junior, Mid, Senior or Lead with **65.2% accuracy** (guessing gives 30.0%); **97.9%** of predictions are within one level.
- **Market value regressor (XGBoost)** predicts yearly salary with **$R^2 = 0.649$** on log salary; MAE **$27,810** and RMSE **$52,049**, against $49,222 and $74,183 for a no-ML baseline.
- A **growth simulator** moves the user forward month by month to estimate the time to a goal.
- A **hybrid recommender** combines a 52-topic knowledge base, the ML what-if engine and five years of market trends into a learning roadmap with free resources.

The system is delivered as a **FastAPI** backend and a **Next.js** web application with a 3D **Three.js** interface.

3.2 Problem statement

Developers, especially juniors, cannot easily tell where they stand or what to learn next. A developer wants to know three things:

1. **Where am I?** Junior, Mid, Senior or Lead, and what is my market value?
2. **How long to my goal?** At my study pace, when do I reach Senior or a target salary?
3. **What should I learn next?** Which skills raise my level and pay, and which workplace skills am I missing?

DevAscend answers these questions with data from real developers instead of opinion. No public dataset follows the same developers for many years, so the project learns from developers at different career stages and **simulates** one person's future.

3.3 Objectives

1. Store five years of survey data in Apache Cassandra and train from it.
2. Clean the data and define a fair Junior / Mid / Senior / Lead label.
3. Train and compare three algorithms for each prediction task, and keep the best.
4. Measure accuracy honestly on unseen data, including how far predictions are from actual values.
5. Simulate growth month by month and estimate the time to a goal.

6. Recommend skills, including ones the survey does not cover (Tailwind CSS, validation, system design), with free learning resources.
7. Deliver everything as a web application with an API and a futuristic 3D interface.

3.4 Scope

Inputs: years of coding, years of professional experience, role, skills, education, country, study hours per week and a goal (a level or a salary).

Outputs: level, 0–100 score and model confidence; market value with an 80% range; 3-year growth curve and time to goal; skill gaps and what-if boosts; learning guide and roadmap; a model accuracy page.

3.5 System architecture

```
OFFLINE (training pipeline)
Survey CSVs 2021-2025 --ingest.py--> Apache Cassandra (survey_responses)
      --clean.py--> clean data --train.py--> XGBoost models (.joblib)
      --evaluate.py / trends.py--> evaluation.json, trends.json

ONLINE (application)
Browser (Next.js + React + Three.js)
      |  HTTP / JSON
      v
FastAPI backend (api/main.py) --> ML layer (predictor.py, recommender.py, in memory)
      |
      v
Apache Cassandra (predictions table: prediction history)
```

The browser talks only to the FastAPI backend. The backend runs the ML layer in memory and uses Cassandra for prediction history. If Cassandra is down, predictions still work and only the history is disabled.

3.6 Dataset

Source: Stack Overflow Developer Survey 2021, 2022, 2023, 2024 and 2025 (ODbL licence), downloaded from the official Stack Overflow GitHub repository. The survey changes a little each year (for example, 2025 renamed `YearsCodePro` to `WorkExp`), so `ingest.py` maps every year into one common schema.

Survey column	Meaning	Stored as
YearsCode	total years of coding	years_code
YearsCodePro (2025: WorkExp)	professional years	years_code_pro
DevType	job role(s)	dev_type

Language / Platform / Webframe / Database / ToolsTech / MiscTech HaveWorkedWith	technologies used	six text columns
EdLevel, Country	education, country	ed_level, country
ConvertedCompYearly	yearly salary in USD	salary_usd
AISelect (2023+)	uses AI tools?	ai_select
MainBranch, Employment	professional developer? employed?	filters

4. TOOLS AND TECHNIQUES USED

4.1 Tools (technology stack)

Layer	Tool	Purpose
Language	**Python 3**, JavaScript	Python for data and ML; JavaScript (React) for the website.
Database	**Apache Cassandra** (Docker, Astra DB in the cloud), cassandra-driver	Stores the 360,519 survey responses (training data) and the prediction history.
Data processing	**Pandas**, **NumPy**, PyArrow (Parquet)	Cleaning, transforming and feature building.
Machine learning	**scikit-learn**, **XGBoost**, joblib	Logistic / Linear Regression, Random Forest, XGBoost; metrics; saving models.
Visualisation	Matplotlib, Seaborn, Jupyter Notebook	EDA charts, confusion matrix, feature importance, evaluation plots.
Backend API	**FastAPI**, Uvicorn, Pydantic	REST endpoints, input validation (422 errors), automatic docs at /docs.
Frontend	**Next.js 16**, **React 19**	Two pages: Predictor and Model accuracy; reusable Form and Input components.
Charts (web)	**Recharts**	Growth curve with two scenarios.
3D interface	**Three.js**	Glowing wireframe crystal, orbiting rings, particle star field.
External API	GitHub REST API	"Fetch from GitHub" reads a user's public repositories to fill in skills.
DevOps	**Docker**, Docker Compose; Render, Vercel	Runs Cassandra locally; free cloud deployment of API and website.

4.2 Why Apache Cassandra

Cassandra is a distributed NoSQL database with **no master node**: it scales horizontally and has no single point of failure. Design is **query-first**: there are no JOINs, so each table is shaped around the query it answers. The **partition key** decides which node stores a row; the **clustering key** sorts rows inside a partition.

Table (keyspace devascend)	Primary key	Query it serves

survey_responses	((survey_year), response_id)	all developers of one survey year
predictions	((bucket), created_at DESC, id)	the latest N predictions, newest first

`ingest.py` uses a **prepared INSERT** (parsed once, reused for every row) and runs 100 inserts in parallel: 360,519 rows load in about four minutes. `clean.py` reads one partition per survey year with **paging** (fetch_size = 5000). A `COUNT(*)` over the whole table is a full cluster scan, so counts are taken per partition.

4.3 Data cleaning (clean.py)

Step	Rows left
Raw responses, 5 years	360,519
Professional developers, employed, salary given	163,791
Salary between $3,000 and $600,000 (typos and outliers removed)	156,922
Valid experience ("Less than 1 year" → 0, "More than 50 years" → 51)	155,862
Has a developer role (students, retired, academics removed)	154,797
At least one skill	**153,645**

Other transformations: more than 30 DevType answers are mapped to **15 roles**; education becomes an **ordered number** 0 (primary) to 6 (PhD); skill spellings are unified ("React.js" → "React"); years coding is never below professional years.

4.4 Feature engineering (features.py) — 151 features

Group	Columns	Encoding
years_code, years_pro, ed_level, survey_year, num_skills	5	numbers
Role	15	one-hot
Country (top 30 + Other)	31	one-hot
Skills (100 most common)	100	multi-hot (1 = uses it)

Training and the API share the same `build_features` function, so a user's profile is encoded exactly like the training data.

4.5 The level label

The survey has no "Junior/Senior" question, so the label is defined from two facts: **level_score = 100 × (experience percentile + salary percentile within the same country and year) / 2**. Computing the salary percentile within each country and year makes the label fair across countries and inflation. The score is cut into four levels:

Level	Share	level_score range	Median professional years	Median salary
Junior	30%	below 34.9	3	$36,410
Mid	30%	34.9 – 59.2	7	$62,820
Senior	25%	59.2 – 77.7	12	$90,647
Lead	15%	77.7 and above	20	$139,216

4.6 Machine learning techniques

- **Train/test split:** 80% train (122,916) / 20% test (30,729), stratified by level, random_state = 42; all six models use the same split for a fair comparison.
- **Model A (classification, 4 classes):** best model chosen by macro F1.
- **Model B (regression):** learns **log(salary)** because salaries are right-skewed; predictions are converted back with exp().
- **Monotone constraint:** XGBoost may only predict a higher salary when experience rises, building domain knowledge into the model.

Algorithm	How it works	Settings
Logistic / Linear Regression	weighted sum of the features; linear baseline	StandardScaler first; C = 0.5 for logistic
Random Forest	150 decision trees on random samples, combined by voting / averaging (bagging)	max depth 18, min 5 samples per leaf
XGBoost	trees built one after another, each correcting previous errors (gradient boosting)	500–800 trees, depth 6–7, learning rate 0.06–0.08

4.7 Evaluation metrics

Classification: Accuracy, F1 (macro), within ±1 level, confusion matrix. **Regression:** MAE = $(1/n)\ \Sigma\ |\hat{y} - y|$; RMSE = $\sqrt{((1/n)\ \Sigma\ (\hat{y} - y)^2)}$; $R^2 = 1 - \Sigma(\hat{y} - y)^2 / \Sigma(y - \bar{y})^2$. Both models are compared with a no-ML baseline.

4.8 Growth simulator and what-if engine (Model C)

1. For month m = 0 … 120, experience = starting value + m/12 years.

2. Add the next roadmap skill once its study hours are done (10 h/week ≈ 43 h/month; topic hours from the knowledge base).
3. Predict level probabilities (Model A) and salary (Model B) for all 121 months in one batch.
4. The goal is reached at the first month where P(level ≥ goal) ≥ 50%, or predicted salary ≥ target.

Two scenarios are simulated (experience only, and with the roadmap); the gap between them is the value of learning. **What-if:** each missing skill is switched on, both models are re-run, and the change in value and level score is measured, a simple model explanation similar in spirit to SHAP.

4.9 Hybrid recommender (recommender.py)

Source	Contribution	Example
Knowledge base (52 topics, learning_topics.json)	prerequisites, role, level, free resources	HTML/CSS unlocks Tailwind CSS
ML what-if engine	predicted market value boost	TypeScript: +7.9%
Market trends (trends.py, 5 survey years from Cassandra)	rising or declining skills	TypeScript 38% → 50%

Combining knowledge with data is the standard answer to the **cold-start problem** (the survey knows only ~100 technologies).

5. CODE

5.1 Project structure

```
Prediction Model/
├── docker-compose.yml        Apache Cassandra (one node)
├── ml/
│   ├── ingest.py             survey CSVs -> Cassandra
│   ├── cassandra_db.py       connection + schema
│   ├── clean.py              read from Cassandra, clean, level label
│   ├── features.py           151 features (build_features)
│   ├── train.py              3 algorithms x 2 tasks, pick the best, save
│   ├── evaluate.py           test-set report vs baseline
│   ├── trends.py             skill trends 2021-2025
│   ├── predictor.py          predict, growth simulator, what-if
│   ├── recommender.py        hybrid learning guide
│   └── knowledge/learning_topics.json   52 topics
├── api/
│   ├── main.py               FastAPI endpoints
│   ├── schemas.py            Pydantic validation
│   └── db.py                 prediction history in Cassandra
└── web/                      Next.js + React + Three.js
    ├── app/page.js           Predictor page
    ├── app/accuracy/page.js  Model accuracy page
    ├── components/           Form, Input, GrowthChart, Roadmap, Scene3D ...
    └── lib/api.js            calls to the API
```

5.2 Docker Compose (Cassandra)

docker-compose.yml

```
# Runs Apache Cassandra locally for the DevAscend project.
# Start:  docker compose up -d
# Stop:   docker compose down        (data is kept in the "cassandra_data" volume)
services:
  cassandra:
    image: cassandra:4.1
    container_name: devascend-cassandra
    ports:
      - "9042:9042"          # CQL port used by the Python driver
    environment:
      CASSANDRA_CLUSTER_NAME: devascend
      MAX_HEAP_SIZE: 1G      # keep memory usage laptop-friendly
      HEAP_NEWSIZE: 256M
    volumes:
      - cassandra_data:/var/lib/cassandra
    healthcheck:
      test: ["CMD-SHELL", "cqlsh -e 'describe keyspaces' || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 10

volumes:
  cassandra_data:
```

5.3 Cassandra connection and schema

ml/cassandra_db.py

```
"""
Small helper for talking to Cassandra.

Cassandra basics used in this project:
  * KEYSPACE  = like a "database" in MySQL. Ours is called `devascend`.
  * TABLE     = rows + columns, but every table needs a PRIMARY KEY.
  * PARTITION KEY (first part of the primary key) decides which node stores the row.
    We partition survey rows by `survey_year`, so "give me all 2024 rows" is one fast query.
  * CLUSTERING KEY (rest of the primary key) sorts rows inside a partition.

Two places Cassandra can live:
  * LOCAL  - Docker on your laptop (docker-compose.yml), used for development and training.
  * CLOUD  - DataStax Astra DB (managed Apache Cassandra), used when the app is deployed.
             Activated by the ASTRA_DB_* environment variables (see config.py / docs/10-
deployment.md).
"""
import base64
import tempfile
import time
from pathlib import Path

from cassandra.cluster import Cluster

from config import (ASTRA_DB_SECURE_BUNDLE, ASTRA_DB_SECURE_BUNDLE_B64, ASTRA_DB_TOKEN,
                    CASSANDRA_HOSTS, CASSANDRA_PORT, KEYSPACE)

USING_ASTRA = bool(ASTRA_DB_TOKEN)


def _secure_bundle_path() -> str:
    """Astra needs its 'secure connect bundle' zip as a FILE. Hosting secrets are text, so we
    also accept the zip as base64 and write it to a temporary file."""
    if ASTRA_DB_SECURE_BUNDLE:
        return ASTRA_DB_SECURE_BUNDLE
    if ASTRA_DB_SECURE_BUNDLE_B64:
        path = Path(tempfile.gettempdir()) / "astra_secure_connect_bundle.zip"
        path.write_bytes(base64.b64decode(ASTRA_DB_SECURE_BUNDLE_B64))
        return str(path)
    raise RuntimeError("ASTRA_DB_TOKEN is set but no secure connect bundle was given "
                       "(set ASTRA_DB_SECURE_BUNDLE or ASTRA_DB_SECURE_BUNDLE_B64)")


def make_cluster(connect_timeout: int = 10) -> Cluster:
    """A Cluster object for Astra (cloud) or the local Docker Cassandra."""
    if USING_ASTRA:
        from cassandra.auth import PlainTextAuthProvider

        # Astra login: the user name is literally "token", the password is the AstraCS:... token
        return Cluster(cloud={"secure_connect_bundle": _secure_bundle_path()},
                       auth_provider=PlainTextAuthProvider("token", ASTRA_DB_TOKEN),
                       connect_timeout=connect_timeout)
    return Cluster(CASSANDRA_HOSTS, port=CASSANDRA_PORT, connect_timeout=connect_timeout)

SCHEMA = [
    f"""
    CREATE KEYSPACE IF NOT EXISTS {KEYSPACE}
    WITH replication = {{'class': 'SimpleStrategy', 'replication_factor': 1}}
    """,
    # Raw survey answers (one row = one developer). Query pattern: all rows of a given year.
```

```
    f"""
    CREATE TABLE IF NOT EXISTS {KEYSPACE}.survey_responses (
        survey_year     int,
        response_id     int,
        main_branch     text,
        employment      text,
        country         text,
        ed_level        text,
        years_code      text,
        years_code_pro  text,
        dev_type        text,
        languages       text,
        platforms       text,
        webframes       text,
        databases       text,
        tools           text,
        misc_tech       text,
        ai_select       text,
        salary_usd      double,
        PRIMARY KEY ((survey_year), response_id)
    )
    """,
    # Every prediction made by the web app. Query pattern: latest N predictions.
    # All rows share bucket='all' and are sorted newest-first by created_at.
    f"""
    CREATE TABLE IF NOT EXISTS {KEYSPACE}.predictions (
        bucket        text,
        created_at    timestamp,
        id            uuid,
        role          text,
        country       text,
        years_pro     double,
        level         text,
        score         double,
        salary_usd    double,
        goal          text,
        months_to_goal int,
        profile_json  text,
        PRIMARY KEY ((bucket), created_at, id)
    ) WITH CLUSTERING ORDER BY (created_at DESC, id ASC)
    """,
]


def connect(retries: int = 20, wait_seconds: int = 6):
    """Connect to Cassandra, retrying while the container is still booting."""
    last_error = None
    for attempt in range(1, retries + 1):
        try:
            cluster = make_cluster()
            session = cluster.connect()
            return cluster, session
        except Exception as exc:  # NoHostAvailable while Cassandra starts up
            last_error = exc
            print(f"  Cassandra not ready (attempt {attempt}/{retries}), retrying in
{wait_seconds}s...")
            time.sleep(wait_seconds)
    raise RuntimeError(
        "Could not connect to Cassandra. Is it running?  ->  docker compose up -d"
    ) from last_error


def create_schema(session):
    for statement in SCHEMA:
```

```
        if USING_ASTRA and "CREATE KEYSPACE" in statement:
            continue  # on Astra the keyspace is created in the Astra web console, not with CQL
        session.execute(statement)
    session.set_keyspace(KEYSPACE)
    # Columns added after the first version of the table (ALTER fails harmlessly if it already
exists)
    try:
        session.execute("ALTER TABLE survey_responses ADD ai_select text")
    except Exception:
        pass
```

5.4 Loading the survey into Cassandra

ml/ingest.py

```
"""
STEP 1 — Load the raw Stack Overflow survey CSVs into Cassandra.

    python ingest.py

Reads ml/data/raw/survey_<year>.csv for every year in config.SURVEY_YEARS,
keeps only the columns we need, maps them to ONE common schema and inserts
them into the Cassandra table `devascend.survey_responses`.

Why a common schema? The survey changes a little every year. Example: 2025
renamed "YearsCodePro" to "WorkExp" and moved Docker from "ToolsTech" to "Platform".
We store all years in the same table so later steps don't care about the year.
"""
import math
import time

import pandas as pd
from cassandra.concurrent import execute_concurrent_with_args

from cassandra_db import connect, create_schema
from config import RAW_DIR, SKILL_COLUMNS, SURVEY_YEARS

# survey column name -> our Cassandra column name
BASE_COLUMNS = {
    "ResponseId": "response_id",
    "MainBranch": "main_branch",
    "Employment": "employment",
    "Country": "country",
    "EdLevel": "ed_level",
    "YearsCode": "years_code",
    "YearsCodePro": "years_code_pro",
    "WorkExp": "years_code_pro",          # 2025 name for professional experience
    "DevType": "dev_type",
    "ConvertedCompYearly": "salary_usd",
    "AISelect": "ai_select",               # "do you use AI tools?" (asked from 2023)
}
SKILL_RENAME = {survey_col: our_col for our_col, survey_col in SKILL_COLUMNS.items()}
TEXT_COLUMNS = [
    "main_branch", "employment", "country", "ed_level", "years_code", "years_code_pro",
    "dev_type", *SKILL_COLUMNS.keys(), "ai_select",
]

INSERT_CQL = f"""
INSERT INTO survey_responses (survey_year, response_id, {", ".join(TEXT_COLUMNS)}, salary_usd)
VALUES ({", ".join(["?"] * (len(TEXT_COLUMNS) + 3))})
"""
```

```
def read_year(year: int) -> pd.DataFrame:
    path = RAW_DIR / f"survey_{year}.csv"
    if not path.exists():
        raise FileNotFoundError(f"Missing {path}. Download the survey CSV (see README).")

    header = pd.read_csv(path, nrows=0, encoding="utf-8-sig").columns
    wanted = {**BASE_COLUMNS, **SKILL_RENAME}
    # 2024 and earlier have YearsCodePro; ignore WorkExp there (it means something else).
    if "YearsCodePro" in header:
        wanted.pop("WorkExp")
    usecols = [c for c in wanted if c in header]

    df = pd.read_csv(path, usecols=usecols, encoding="utf-8-sig", low_memory=False, dtype=str)
    df = df.rename(columns=wanted)
    for col in TEXT_COLUMNS:                      # columns a year doesn't have -> empty
        if col not in df.columns:
            df[col] = None
    df["survey_year"] = year
    df["response_id"] = df["response_id"].astype(int)
    df["salary_usd"] = pd.to_numeric(df["salary_usd"], errors="coerce")
    return df


def to_rows(df: pd.DataFrame):
    """Convert a DataFrame into plain Python tuples in the INSERT column order."""
    cols = ["survey_year", "response_id", *TEXT_COLUMNS, "salary_usd"]
    for record in df[cols].itertuples(index=False, name=None):
        yield tuple(
            None if (isinstance(v, float) and math.isnan(v)) else v
            for v in record
        )


def main():
    print("Connecting to Cassandra...")
    cluster, session = connect()
    create_schema(session)
    insert = session.prepare(INSERT_CQL)   # prepared once, executed many times (fast)

    total = 0
    for year in SURVEY_YEARS:
        start = time.time()
        df = read_year(year)
        results = execute_concurrent_with_args(
            session, insert, list(to_rows(df)), concurrency=100, raise_on_first_error=True
        )
        ok = sum(1 for success, _ in results if success)
        total += ok
        print(f"  {year}: inserted {ok:,} rows in {time.time() - start:.1f}s")

    # Count per partition (one year at a time). A COUNT(*) over the whole table is a full
    # cluster scan, which Cassandra handles badly — querying by partition key is the Cassandra way.
    count = sum(
        session.execute("SELECT COUNT(*) FROM survey_responses WHERE survey_year = %s", (year,),
timeout=120).one()[0]
        for year in SURVEY_YEARS
    )
    print(f"Done. Inserted {total:,} rows. Table now holds {count:,} rows.")
    cluster.shutdown()
```

```
if __name__ == "__main__":
    main()
```

5.5 Cleaning and the level label

ml/clean.py

```
"""
STEP 2 — Read raw survey rows from Cassandra, clean them, create the level labels.

    python clean.py                 # read from Cassandra (normal way)
    python clean.py --source csv    # read the CSV files directly (backup if Cassandra is down)

Output: ml/data/clean.parquet  (one row per developer, ready for training)

Cleaning steps (each one is printed with the number of rows left):
  1. keep professional developers who are employed and reported a salary
  2. remove salary outliers (< $3k or > $600k per year)
  3. convert experience text to numbers ("Less than 1 year" -> 0)
  4. map the many DevType answers to ~15 simple roles
  5. merge all technology columns into one clean skill list
  6. create the target label: Junior / Mid / Senior / Lead
"""
import argparse

import numpy as np
import pandas as pd

from config import CLEAN_FILE, KEYSPACE, LEVELS, MAX_SALARY, MIN_SALARY, SKILL_COLUMNS, SURVEY_YEARS
from features import combine_skills, education_to_number, normalize_role, years_to_number


# ---------------- Loading ----------------
def load_from_cassandra() -> pd.DataFrame:
    from cassandra.query import SimpleStatement

    from cassandra_db import connect

    cluster, session = connect()
    session.set_keyspace(KEYSPACE)
    frames = []
    for year in SURVEY_YEARS:
        # One partition per year -> efficient query. fetch_size = rows per network page.
        query = SimpleStatement("SELECT * FROM survey_responses WHERE survey_year = %s",
fetch_size=5000)
        rows = list(session.execute(query, (year,), timeout=120))
        frames.append(pd.DataFrame(rows))
        print(f"  loaded {len(rows):,} rows for {year} from Cassandra")
    cluster.shutdown()
    return pd.concat(frames, ignore_index=True)


def load_from_csv() -> pd.DataFrame:
    from ingest import read_year

    frames = [read_year(y) for y in SURVEY_YEARS]
    return pd.concat(frames, ignore_index=True)


# ---------------- Cleaning ----------------
def clean(raw: pd.DataFrame) -> pd.DataFrame:
```

```
    df = raw.copy()
    print(f"Start: {len(df):,} rows")

    # 1. professional developers with a job and a salary
    df = df[df["main_branch"].fillna("").str.contains("developer by profession")]
    df = df[df["employment"].fillna("").str.contains("Employed|Independent", regex=True)]
    df = df[df["salary_usd"].notna()]
    print(f"  professional + employed + has salary: {len(df):,}")

    # 2. salary outliers
    df = df[(df["salary_usd"] >= MIN_SALARY) & (df["salary_usd"] <= MAX_SALARY)]
    print(f"  salary between ${MIN_SALARY:,} and ${MAX_SALARY:,}: {len(df):,}")

    # 3. experience text -> numbers
    df["years_code"] = df["years_code"].map(years_to_number)
    df["years_pro"] = df["years_code_pro"].map(years_to_number)
    df = df.dropna(subset=["years_code", "years_pro"])
    df = df[df["years_pro"] <= 50]
    # you can't have more professional years than total coding years
    df["years_code"] = np.maximum(df["years_code"], df["years_pro"])
    print(f"  valid experience: {len(df):,}")

    # 4. role
    df["role"] = df["dev_type"].map(normalize_role)
    df = df.dropna(subset=["role"])
    print(f"  has a developer role: {len(df):,}")

    # 5. education + skills
    df["ed_level"] = df["ed_level"].map(education_to_number)
    df["skills"] = df[list(SKILL_COLUMNS.keys())].apply(combine_skills, axis=1)
    df = df[df["skills"] != ""]
    df["country"] = df["country"].fillna("Other")
    print(f"  has at least one skill: {len(df):,}")

    # 6. level label
    df = add_level_label(df)

    # AI usage answer (only asked from 2023) -> True / False / missing. Used for the AI trend.
    if "ai_select" not in df.columns:
        df["ai_select"] = None
    df["uses_ai"] = df["ai_select"].map(
        lambda v: None if not isinstance(v, str) else v.startswith("Yes"))

    keep = ["survey_year", "response_id", "years_code", "years_pro", "ed_level", "role",
            "country", "skills", "salary_usd", "level_score", "level", "uses_ai"]
    return df[keep].reset_index(drop=True)


def add_level_label(df: pd.DataFrame) -> pd.DataFrame:
    """
    The survey has no "Junior/Senior" question, so we DEFINE the level from two facts:
      * experience percentile  — how experienced you are compared with all developers
      * salary percentile      — how well the market pays you compared with developers
                                 in the SAME country and SAME year (fair across countries/inflation)
    level_score = average of the two (0..100). Then:
        bottom 30% -> Junior,  next 30% -> Mid,  next 25% -> Senior,  top 15% -> Lead
    """
    country_counts = df["country"].value_counts()
    big_country = df["country"].where(df["country"].map(country_counts) >= 300, "Other")

    exp_pct = df["years_pro"].rank(pct=True)
    sal_pct = df.groupby([big_country, df["survey_year"]])["salary_usd"].rank(pct=True)
    df["level_score"] = (100 * (exp_pct + sal_pct) / 2).round(2)
```

```
    cuts = df["level_score"].quantile([0.30, 0.60, 0.85]).to_list()
    df["level"] = pd.cut(df["level_score"], bins=[-1, *cuts, 101], labels=LEVELS).astype(str)
    print(f"  level cut-offs on level_score: {[round(c, 1) for c in cuts]}")
    print("  level counts:", df["level"].value_counts().reindex(LEVELS).to_dict())
    return df


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source", choices=["cassandra", "csv"], default="cassandra")
    args = parser.parse_args()

    print(f"Loading raw data from {args.source}...")
    raw = load_from_cassandra() if args.source == "cassandra" else load_from_csv()
    df = clean(raw)
    CLEAN_FILE.parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(CLEAN_FILE, index=False)
    print(f"Saved {len(df):,} clean rows -> {CLEAN_FILE}")
    print(df.groupby("level")[["years_pro", "salary_usd"]].median().reindex(LEVELS).round(0))


if __name__ == "__main__":
    main()
```

5.6 Feature engineering

ml/features.py (build_features)

```
                    skills.add(s)
    return ";".join(sorted(skills))


# ---------------- Profile(s) -> model input matrix ----------------
NUMERIC_FEATURES = ["years_code", "years_pro", "ed_level", "survey_year", "num_skills"]


def build_features(df: pd.DataFrame, meta: dict) -> pd.DataFrame:
    """
    df needs columns: years_code, years_pro, ed_level, survey_year, role, country, skills (';'
string).
    meta holds the vocabulary decided at training time: meta["roles"], meta["countries"],
meta["skills"].
    Returns a numeric DataFrame whose columns are ALWAYS meta["feature_columns"] (same order).
    """
    out = pd.DataFrame(index=df.index)
    out["years_code"] = df["years_code"].astype(float)
    out["years_pro"] = df["years_pro"].astype(float)
    out["ed_level"] = df["ed_level"].astype(float)
    out["survey_year"] = df["survey_year"].astype(float)

    # Skills -> one 0/1 column per known skill ("multi-hot encoding")
    skill_dummies = (
        df["skills"].fillna("").astype(str).str.get_dummies(sep=";")
        .reindex(columns=meta["skills"], fill_value=0)
    )
    out["num_skills"] = skill_dummies.sum(axis=1).astype(float)

    # Role and country -> one-hot columns (unknown country -> "Other")
    country = df["country"].where(df["country"].isin(meta["countries"]), "Other")
    role = df["role"].where(df["role"].isin(meta["roles"]), "Other")
    role_dummies = pd.get_dummies(role).reindex(columns=meta["roles"], fill_value=0)
```

```
    country_dummies = pd.get_dummies(country).reindex(columns=meta["countries"], fill_value=0)
    role_dummies.columns = [f"role_{c}" for c in role_dummies.columns]
    country_dummies.columns = [f"country_{c}" for c in country_dummies.columns]
    skill_dummies.columns = [f"skill_{c}" for c in skill_dummies.columns]

    X = pd.concat([out, role_dummies, country_dummies, skill_dummies], axis=1).astype(float)
    if "feature_columns" in meta:
        X = X.reindex(columns=meta["feature_columns"], fill_value=0.0)
    return X


def profiles_to_frame(profiles: list[dict]) -> pd.DataFrame:
    """List of profile dicts (from the API) -> DataFrame accepted by build_features."""
    rows = []
    for p in profiles:
        rows.append({
            "years_code": max(float(p["years_code"]), float(p["years_pro"])),
            "years_pro": float(p["years_pro"]),
            "ed_level": int(p["ed_level"]),
            "survey_year": int(p["survey_year"]),
            "role": p["role"],
            "country": p["country"],
            "skills": ";".join(sorted(set(p["skills"]))),
        })
    return pd.DataFrame(rows)
```

5.7 Training and model comparison

ml/train.py (algorithms and training)

```
def monotone_constraints(columns):
    """+1 = prediction may only go UP when this feature goes up (more experience never hurts)."""
    return "(" + ",".join("1" if c in ("years_code", "years_pro") else "0" for c in columns) + ")"


def classifiers(columns, fast):
    n = 60 if fast else 150
    return {
        "Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000,
C=0.5)),
        "Random Forest": RandomForestClassifier(
            n_estimators=n, max_depth=18, min_samples_leaf=5, max_samples=0.6,
            n_jobs=-1, random_state=RANDOM_STATE),
        "XGBoost": XGBClassifier(
            n_estimators=200 if fast else 500, max_depth=6, learning_rate=0.08,
            subsample=0.8, colsample_bytree=0.8, tree_method="hist", n_jobs=-1,
            random_state=RANDOM_STATE),
    }


def regressors(columns, fast):
    n = 60 if fast else 150
    return {
        "Linear Regression": make_pipeline(StandardScaler(), LinearRegression()),
        "Random Forest": RandomForestRegressor(
            n_estimators=n, max_depth=18, min_samples_leaf=5, max_samples=0.6,
            n_jobs=-1, random_state=RANDOM_STATE),
        "XGBoost": XGBRegressor(
            n_estimators=300 if fast else 800, max_depth=7, learning_rate=0.06,
            subsample=0.8, colsample_bytree=0.8, tree_method="hist", n_jobs=-1,
            monotone_constraints=monotone_constraints(columns), random_state=RANDOM_STATE),
```

```
    }


# ---------------- 3. Training + evaluation ----------------
def train_level_models(X_train, X_test, y_train, y_test, fast):
    print("\n=== Model A: Level Classifier ===")
    results, fitted = [], {}
    for name, model in classifiers(X_train.columns, fast).items():
        start = time.time()
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        row = {
            "model": name,
            "accuracy": round(accuracy_score(y_test, pred), 4),
            "f1_macro": round(f1_score(y_test, pred, average="macro"), 4),
            # "almost right": predicted level is the true level or one step away
            "within_one_level": round(float(np.mean(np.abs(np.asarray(pred) - np.asarray(y_test)) <=
1)), 4),
            "train_seconds": round(time.time() - start, 1),
        }
        results.append(row)
        fitted[name] = model
        print(f"  {name:<20} accuracy={row['accuracy']:.3f}  F1={row['f1_macro']:.3f}  "
              f"within-one-level={row['within_one_level']:.3f}  ({row['train_seconds']}s)")
    best = max(results, key=lambda r: r["f1_macro"])["model"]
    print(f"  -> best: {best}")
    return fitted, results, best


def train_value_models(X_train, X_test, y_train, y_test, fast):
    """Salary is very skewed, so the models learn log(salary) and we convert back with exp()."""
    print("\n=== Model B: Market Value Regressor ===")
    results, fitted = [], {}
    for name, model in regressors(X_train.columns, fast).items():
        start = time.time()
        model.fit(X_train, np.log(y_train))
        pred = np.exp(model.predict(X_test))
        row = {
            "model": name,
            "r2_log": round(r2_score(np.log(y_test), np.log(pred)), 4),
            "r2": round(r2_score(y_test, pred), 4),
            "mae_usd": round(mean_absolute_error(y_test, pred), 0),
            "rmse_usd": round(float(np.sqrt(mean_squared_error(y_test, pred))), 0),
            "train_seconds": round(time.time() - start, 1),
        }
        results.append(row)
        fitted[name] = model
        print(f"  {name:<20} R2(log)={row['r2_log']:.3f}  R2={row['r2']:.3f}  "
              f"MAE=${row['mae_usd']:,.0f}  RMSE=${row['rmse_usd']:,.0f}  ({row['train_seconds']}s)")
    best = max(results, key=lambda r: r["r2_log"])["model"]
    print(f"  -> best: {best}")
    return fitted, results, best
```

5.8 Prediction and growth simulation

ml/predictor.py (predictor class)

```
class DevAscendPredictor:
    def __init__(self, models_dir=MODELS_DIR):
        self.level_model = joblib.load(models_dir / "level_model.joblib")
        self.value_model = joblib.load(models_dir / "value_model.joblib")
```

```
        with open(models_dir / "meta.json", encoding="utf-8") as f:
            self.meta = json.load(f)
        self.levels = self.meta["levels"]
        self.known_skills = set(self.meta["skills"])
        self.guide = LearningGuide()   # knowledge base + market trends

    # ------------------------------------------------------------------ helpers
    def _prepare(self, profile: dict) -> dict:
        p = dict(profile)
        skills = (SKILL_ALIASES.get(s.strip(), s.strip()) for s in p.get("skills", []))   # "AWS" ->
full name
        p["skills"] = sorted({s for s in skills if s in self.known_skills})
        p.setdefault("survey_year", LATEST_YEAR)
        return p

    def _raw_batch(self, profiles: list[dict]):
        X = build_features(profiles_to_frame(profiles), self.meta)
        probs = self.level_model.predict_proba(X)                # shape (n, 4)
        log_salary = self.value_model.predict(X)                 # model learned log(salary)
        return probs, log_salary

    def _batch(self, profiles: list[dict]):
        """
        Predict many profiles at once (much faster than one by one).

        The survey only records WHOLE years of experience, so the models change their answer
        only when a year boundary is crossed. To get a smooth month-by-month result we predict
        at the two nearest whole years and blend them (linear interpolation).
        Example: 2.25 years = 75% of the prediction at 2 years + 25% of the prediction at 3 years.
        """
        lower, upper, weights = [], [], []
        for p in profiles:
            frac = p["years_pro"] - math.floor(p["years_pro"])
            lower.append({**p, "years_pro": p["years_pro"] - frac, "years_code": p["years_code"] -
frac})
            upper.append({**p, "years_pro": p["years_pro"] - frac + 1, "years_code": p["years_code"]
- frac + 1})
            weights.append(frac)
        w = np.array(weights)
        if not w.any():   # all whole years -> no blending needed
            probs, log_salary = self._raw_batch(profiles)
        else:
            probs_all, log_all = self._raw_batch(lower + upper)
            n = len(profiles)
            probs = probs_all[:n] * (1 - w[:, None]) + probs_all[n:] * w[:, None]
            log_salary = log_all[:n] * (1 - w) + log_all[n:] * w
        salary = np.exp(log_salary)
        # score 0..100 = probability-weighted typical level_score of each level
        # (e.g. 100% Junior -> 20, 100% Lead -> 85, 50/50 Mid/Senior -> 58)
        score = probs @ np.array(self.meta["level_score_median"])
        return probs, salary, score

    def _goal_reached(self, probs, salary, goal) -> np.ndarray:
        if goal["type"] == "salary":
            return salary >= goal["salary"]
        goal_idx = self.levels.index(goal["level"])
        # reached when the model thinks it is at least 50% likely you are at goal level or above
        return probs[:, goal_idx:].sum(axis=1) >= 0.5

    def _skill_hours(self, skill: str) -> float:
        """Study hours for a skill: from the knowledge base if it has the topic, else the default."""
        topic = self.guide.topic_for(skill)
        return topic["hours"] if topic else HOURS_PER_SKILL
```

```
    def _finish_months(self, plan, hours_per_week):
        """Month in which each planned skill is finished (skills are learned one after another)."""
        if hours_per_week <= 0:
            return [math.inf] * len(plan)
        per_month = hours_per_week * WEEKS_PER_MONTH
        finish, total = [], 0.0
        for skill in plan:
            total += self._skill_hours(skill) / per_month
            finish.append(total)
        return finish

    def _timeline(self, profile, plan, hours, months=MAX_SIM_MONTHS):
        """Profile at month 0..months: +1/12 year experience per month, + each planned skill once
finished."""
        finish = self._finish_months(plan, hours)
        timeline = []
        for m in range(months + 1):
            p = dict(profile)
            p["years_code"] = profile["years_code"] + m / 12
            p["years_pro"] = profile["years_pro"] + m / 12
            # skills the ML model doesn't know (e.g. "Tailwind CSS") take study time but don't change
the prediction
            p["skills"] = list(profile["skills"]) + [s for s, f in zip(plan, finish) if f <= m]
            timeline.append(p)
        return timeline

    def _simulate(self, profile, plan, hours, goal):
        probs, salary, score = self._batch(self._timeline(profile, plan, hours))
        # A developer does not get worse by gaining experience, so we keep the running maximum
        # (smooths small ups and downs that come from the tree models' step-like predictions).
        salary = np.maximum.accumulate(salary)
        score = np.maximum.accumulate(score)
        reached = np.maximum.accumulate(self._goal_reached(probs, salary,
goal).astype(int)).astype(bool)
        month = int(np.argmax(reached)) if reached.any() else None
        return probs, salary, score, month

    # ------------------------------------------------------------------ /predict
    def predict(self, profile: dict) -> dict:
        p = self._prepare(profile)
        probs, salary, score = self._batch([p])
        idx = int(np.argmax(probs[0]))
        # 80% range: on the test set, 80% of real salaries were between low_factor x and
        # high_factor x the prediction (computed in evaluate.py)
        interval = self.meta.get("salary_interval", {"low_factor": 1, "high_factor": 1})
        return {
            "level": self.levels[idx],
            "score": round(float(score[0]), 1),
            "probabilities": {lvl: round(float(v), 3) for lvl, v in zip(self.levels, probs[0])},
            "salary_usd": round(float(salary[0]), -2),
            "salary_range": {
                "low": round(float(salary[0]) * interval["low_factor"], -2),
                "high": round(float(salary[0]) * interval["high_factor"], -2),
                "confidence": 0.8,
            },
            "confidence": round(float(probs[0][idx]), 3),
            "known_skills_used": p["skills"],
        }
```

5.9 Backend API

api/main.py

```
"""
DevAscend API (FastAPI).

Run from the api/ folder:
    uvicorn main:app --reload --port 8000
Interactive docs (try every endpoint in the browser): http://localhost:8000/docs

Endpoints
  GET  /health      is the API up, are the models loaded, is Cassandra connected
  GET  /options     lists for the form (roles, countries, skills, education levels)
  GET  /model-info  accuracy / R² of every algorithm we compared
  GET  /evaluation  detailed test-set errors: MAE, RMSE, R², % error, confusion matrix, examples
  POST /predict     current level, 0-100 score, market value
  POST /growth      growth curve (6m / 1y / 2y / 3y) + time to goal
  POST /analysis    skill gaps, what-if boosts, learning roadmap, learning guide
  GET  /history     latest predictions stored in Cassandra
"""
import json
import os
import sys
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware

# The ML code (features, predictor) lives in ../ml — make it importable.
sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "ml"))

from db import PredictionStore                      # noqa: E402
from predictor import DevAscendPredictor          # noqa: E402
from schemas import PlanRequest, Profile          # noqa: E402

state = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the models ONCE at startup (loading per request would be very slow).
    state["model"] = DevAscendPredictor()
    state["db"] = PredictionStore()
    state["db"].connect()
    yield
    state["db"].close()


app = FastAPI(title="DevAscend API", version="1.0", lifespan=lifespan)

# CORS = which websites may call this API from the browser.
#   * always: any localhost port (development: 3000, 3001 ...)
#   * deployed: the exact site addresses in ALLOWED_ORIGINS, comma-separated,
#     e.g.  ALLOWED_ORIGINS=https://devascend.vercel.app
#   * optional: ALLOWED_ORIGIN_REGEX for Vercel preview links, e.g. https://devascend-.*\.vercel\.app
LOCAL_ORIGINS = r"http://(localhost|127\.0\.0\.1)(:\d+)?"
extra_regex = os.getenv("ALLOWED_ORIGIN_REGEX", "").strip()
app.add_middleware(
    CORSMiddleware,
    allow_origins=[o.strip().rstrip("/") for o in os.getenv("ALLOWED_ORIGINS", "").split(",") if
o.strip()],
    allow_origin_regex=f"{LOCAL_ORIGINS}|{extra_regex}" if extra_regex else LOCAL_ORIGINS,
    allow_methods=["*"],
```

```
    allow_headers=["*"],
)


@app.get("/")
def root():
    """Landing page of the API (what you see when you open the deployed API address)."""
    return {"name": "DevAscend API", "docs": "/docs", "health": "/health"}


def model() -> DevAscendPredictor:
    return state["model"]


def check_profile(profile: Profile) -> dict:
    data = profile.model_dump()
    if data["role"] not in model().meta["roles"]:
        raise HTTPException(422, f"Unknown role '{data['role']}'. Use one of
{model().meta['roles']}")
    if data["years_pro"] > data["years_code"]:
        data["years_code"] = data["years_pro"]
    return data


@app.get("/health")
def health():
    return {"status": "ok", "models_loaded": "model" in state, "cassandra": state["db"].available}


@app.get("/options")
def options():
    meta = model().meta
    skills = sorted(meta["skills"], key=lambda s: -meta["skill_popularity"].get(s, 0))
    # + skills from the knowledge base that the survey doesn't cover (Tailwind CSS, GraphQL...)
    extra = [s for s in model().guide.extra_skill_names() if s not in skills]
    return {
        "roles": meta["roles"],
        "countries": meta["countries"],
        "skills": skills + extra,
        "education_levels": [{"value": int(k), "label": v} for k, v in
meta["education_levels"].items()],
        "levels": meta["levels"],
    }


@app.get("/model-info")
def model_info():
    meta = model().meta
    return {
        "level_model": meta["level_model"],
        "value_model": meta["value_model"],
        "level_results": meta["level_results"],
        "value_results": meta["value_results"],
        "confusion_matrix": meta["confusion_matrix"],
        "feature_importance_level": meta["feature_importance_level"],
        "feature_importance_value": meta["feature_importance_value"],
        "train_rows": meta["train_rows"],
        "test_rows": meta["test_rows"],
    }


@app.get("/evaluation")
def evaluation():
    """Written by ml/evaluate.py: how far the predictions are from the real test data."""
```

```
    path = Path(__file__).resolve().parent.parent / "ml" / "models" / "evaluation.json"
    if not path.exists():
        raise HTTPException(404, "Run `python evaluate.py` in the ml folder first")
    return json.loads(path.read_text(encoding="utf-8"))


@app.post("/predict")
def predict(profile: Profile):
    return model().predict(check_profile(profile))


@app.post("/growth")
def growth(req: PlanRequest):
    profile = check_profile(req.profile)
    goal = req.goal.model_dump()
    result = model().growth(profile, req.hours_per_week, goal, plan=req.plan or None)
    state["db"].save(profile, model().predict(profile), result)   # history in Cassandra
    return result


@app.post("/analysis")
def analysis(req: PlanRequest):
    profile = check_profile(req.profile)
    return model().analysis(profile, req.hours_per_week, req.goal.model_dump())


@app.get("/history")
def history(limit: int = Query(20, ge=1, le=100)):
    return {"cassandra": state["db"].available, "items": state["db"].latest(limit)}
```

5.10 Input validation

api/schemas.py

```
"""
Request/response shapes for the API (Pydantic validates the JSON automatically:
wrong types or out-of-range numbers are rejected with a clear 422 error).
"""
from typing import Literal

from pydantic import BaseModel, Field, model_validator


class Profile(BaseModel):
    years_code: float = Field(ge=0, le=60, description="Total years of coding (incl. learning)")
    years_pro: float = Field(ge=0, le=50, description="Years of professional coding experience")
    role: str = Field(examples=["Full-stack"])
    country: str = Field(examples=["India"])
    ed_level: int = Field(ge=0, le=6, description="0=Primary ... 4=Bachelor, 5=Master, 6=PhD")
    skills: list[str] = Field(default_factory=list, examples=[["JavaScript", "React", "Node.js"]])


class Goal(BaseModel):
    type: Literal["level", "salary"] = "level"
    level: Literal["Junior", "Mid", "Senior", "Lead"] | None = "Senior"
    salary: float | None = Field(default=None, gt=0)

    @model_validator(mode="after")
    def check(self):
        if self.type == "salary" and not self.salary:
            raise ValueError("goal.salary is required when goal.type is 'salary'")
```

```
        if self.type == "level" and not self.level:
            raise ValueError("goal.level is required when goal.type is 'level'")
        return self


class PlanRequest(BaseModel):
    profile: Profile
    hours_per_week: float = Field(default=10, ge=0, le=80)
    goal: Goal = Goal()
    plan: list[str] | None = Field(
        default=None, description="Skills to learn in order. Empty = use the recommended roadmap.")
```

5.11 Frontend: API client

web/lib/api.js

```
// All calls to the FastAPI backend live here, so components never hard-code URLs.
export const API_URL = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000";

async function request(path, body) {
  const res = await fetch(`${API_URL}${path}`, {
    method: body ? "POST" : "GET",
    headers: body ? { "Content-Type": "application/json" } : undefined,
    body: body ? JSON.stringify(body) : undefined,
  });
  if (!res.ok) {
    let detail = res.statusText;
    try {
      const data = await res.json();
      detail = typeof data.detail === "string" ? data.detail : JSON.stringify(data.detail);
    } catch {}
    throw new Error(`${path} failed: ${detail}`);
  }
  return res.json();
}

/**
 * A free hosted API (Hugging Face Space) goes to sleep when nobody uses it and needs some
 * time to wake up. Keep asking /health until it answers; onWaiting() lets the page show
 * "Waking up the server…" meanwhile. Locally it answers on the first try.
 */
export async function waitForApi(onWaiting, { tries = 30, delayMs = 5000 } = {}) {
  for (let i = 0; i < tries; i++) {
    try {
      return await request("/health");
    } catch {
      onWaiting?.(i + 1);
      await new Promise((r) => setTimeout(r, delayMs));
    }
  }
  throw new Error(`Cannot reach the API at ${API_URL}. Is the backend running?`);
}

export const api = {
  health: () => request("/health"),
  options: () => request("/options"),
  modelInfo: () => request("/model-info"),
  evaluation: () => request("/evaluation"),
  history: () => request("/history?limit=10"),
  predict: (profile) => request("/predict", profile),
  analysis: (payload) => request("/analysis", payload),
```

```
  growth: (payload) => request("/growth", payload),
};

export const formatMoney = (v) =>
  v == null ? "—" : `$${Math.round(v).toLocaleString("en-US")}`;

export const LEVEL_COLORS = ["var(--lvl-0)", "var(--lvl-1)", "var(--lvl-2)", "var(--lvl-3)"];
```

5.12 Frontend: reusable components

web/components/Form.jsx

```
"use client";

/**
 * Reusable form wrapper.
 * Handles: title/description, preventing page reload on submit, and a submit button with loading
state.
 *
 * <Form title="..." onSubmit={fn} submitLabel="Analyze" loading={bool}> ...fields... </Form>
 */
export default function Form({ title, description, onSubmit, submitLabel = "Submit", loading = false,
error, children }) {
  function handleSubmit(event) {
    event.preventDefault(); // stop the browser from reloading the page
    onSubmit?.();
  }

  return (
    <form className="form" onSubmit={handleSubmit} noValidate>
      {title && <h2 className="form-title">{title}</h2>}
      {description && <p className="form-desc">{description}</p>}
      {children}
      {error && <div className="alert error" role="alert">{error}</div>}
      <button type="submit" className="btn" disabled={loading}>
        {loading && <span className="spinner" aria-hidden />}
        {loading ? "Predicting…" : submitLabel}
      </button>
    </form>
  );
}
```

web/components/Input.jsx

```
"use client";

/**
 * Reusable input field: label + control + hint/error, for type "text", "number" or "select".
 *
 * <Input label="Years coding" name="years_code" type="number" value={v} onChange={(name, value) =>
...} />
 * <Input label="Role" name="role" type="select" options={[{ value, label }]} ... />
 *
 * onChange receives (name, value) so ONE handler can update any field of a form object.
 * Numbers are converted with Number() so the parent always gets a number for type="number".
 */
export default function Input({
  label, name, type = "text", value, onChange, options = [], hint, error,
  min, max, step, placeholder, required,
}) {
```

```
  const id = `field-${name}`;

  function handleChange(event) {
    const raw = event.target.value;
    onChange?.(name, type === "number" ? (raw === "" ? "" : Number(raw)) : raw);
  }

  return (
    <div className="field">
      {label && <label htmlFor={id}>{label}{required && " *"}</label>}
      {type === "select" ? (
        <select id={id} name={name} className={`input${error ? " invalid" : ""}`} value={value}
onChange={handleChange}>
          {options.map((opt) => {
            const o = typeof opt === "object" ? opt : { value: opt, label: opt };
            return <option key={o.value} value={o.value}>{o.label}</option>;
          })}
        </select>
      ) : (
        <input
          id={id}
          name={name}
          type={type}
          className={`input${error ? " invalid" : ""}`}
          value={value}
          onChange={handleChange}
          min={min}
          max={max}
          step={step}
          placeholder={placeholder}
          aria-invalid={!!error}
        />
      )}
      {error ? <span className="error">{error}</span> : hint && <span className="hint">{hint}</span>}
    </div>
  );
}
```

5. OUTPUT SCREENSHOTS

5.13 Web application

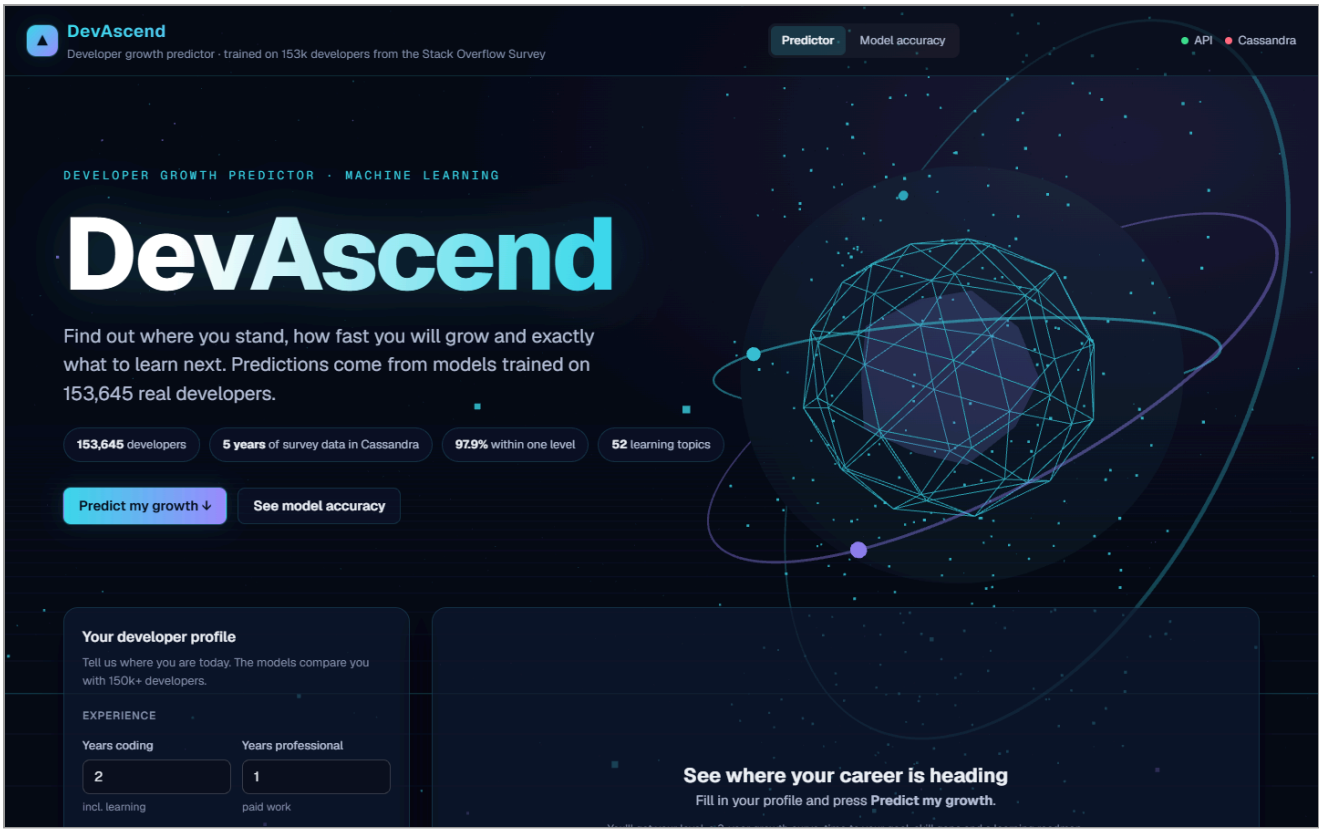


Figure 1: Home page with the 3D Three.js scene and key numbers

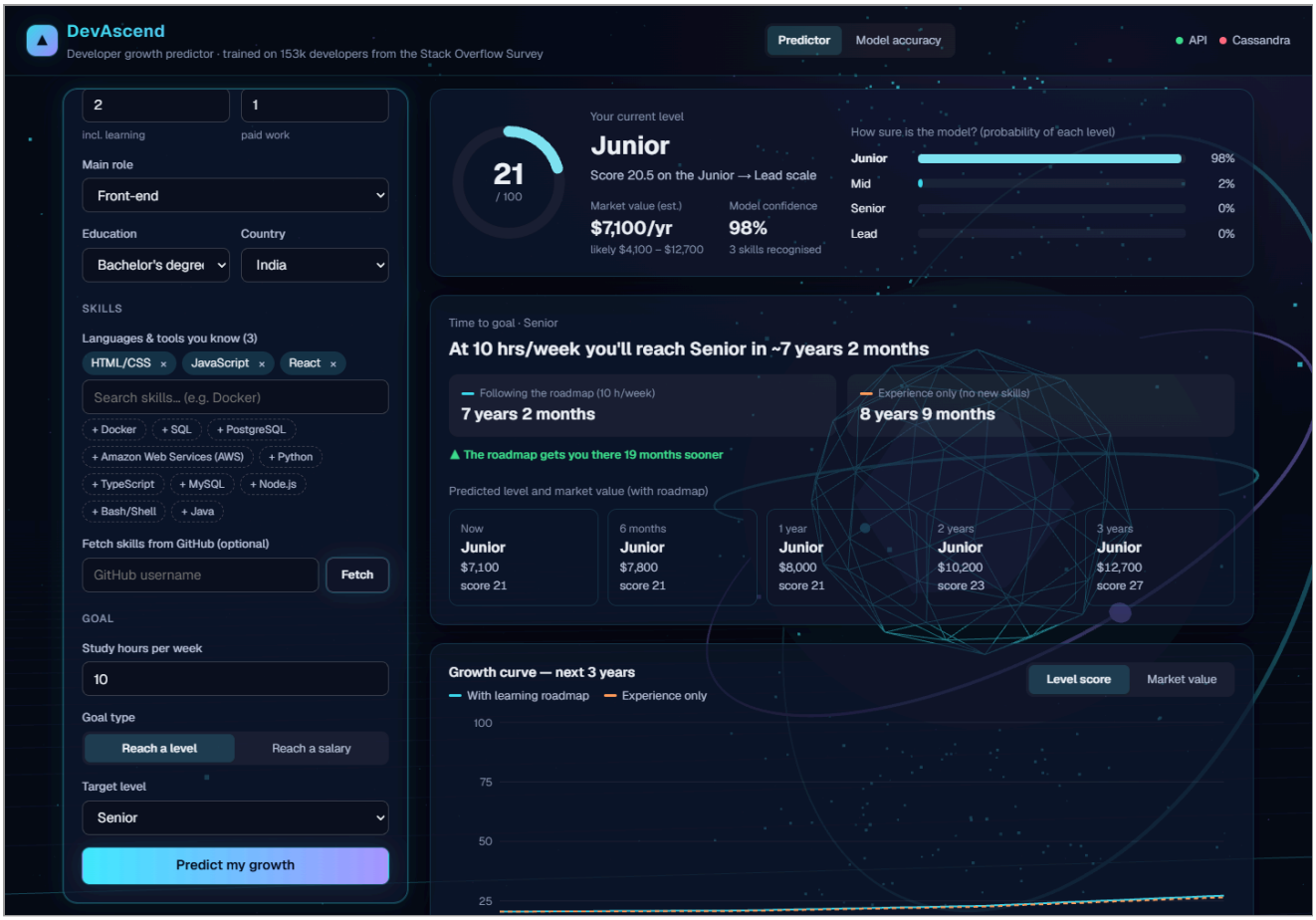


Figure 2: Prediction for a Junior front-end developer in India: level ring, probabilities, market value and time to goal (Senior in ~7 years 2 months with the roadmap vs 8 years 9 months without)

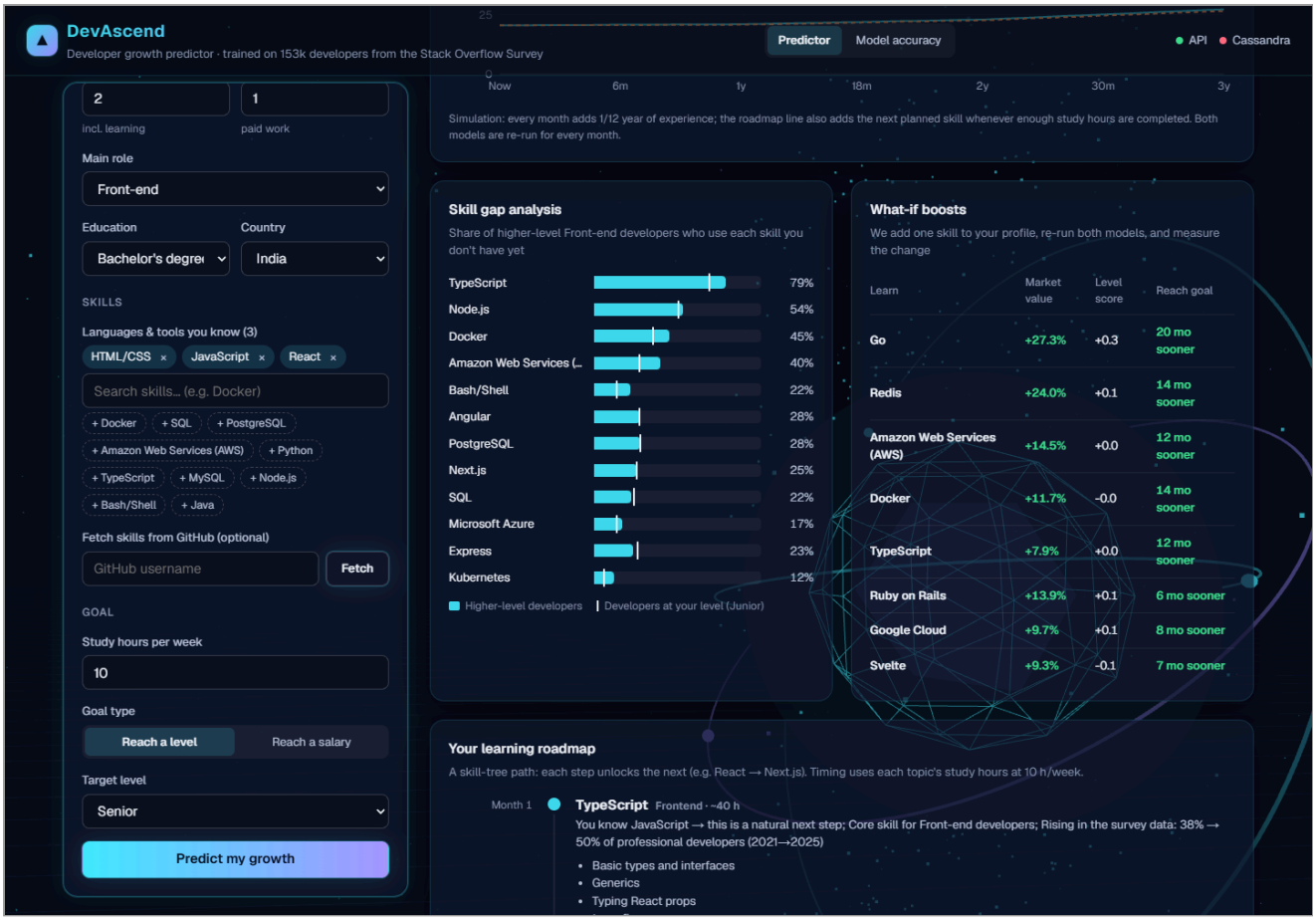


Figure 3: Skill gap analysis and what-if boosts (e.g. Go +27.3% market value, 20 months sooner)

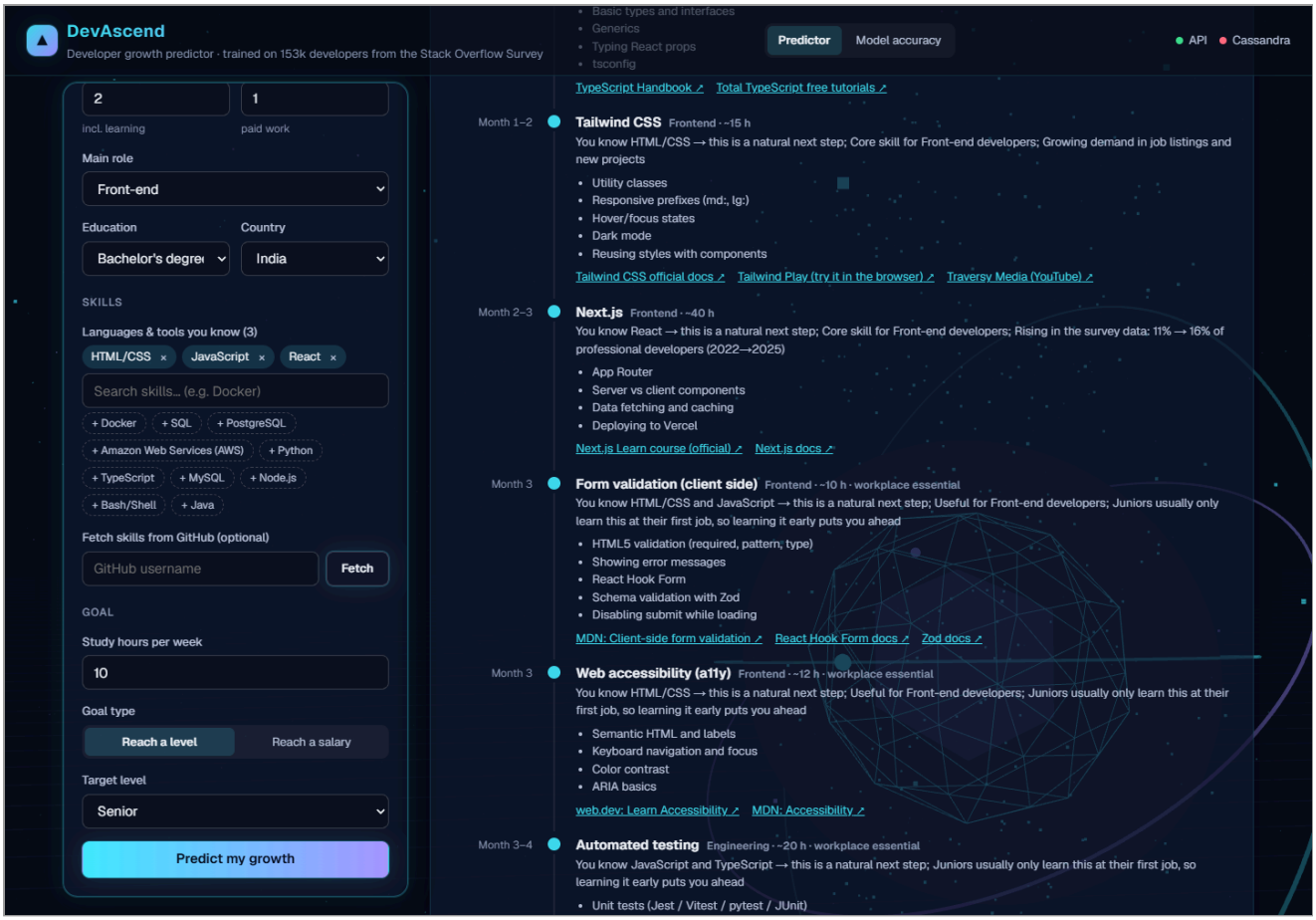


Figure 4: Learning roadmap: a skill-tree path with timing and free resources

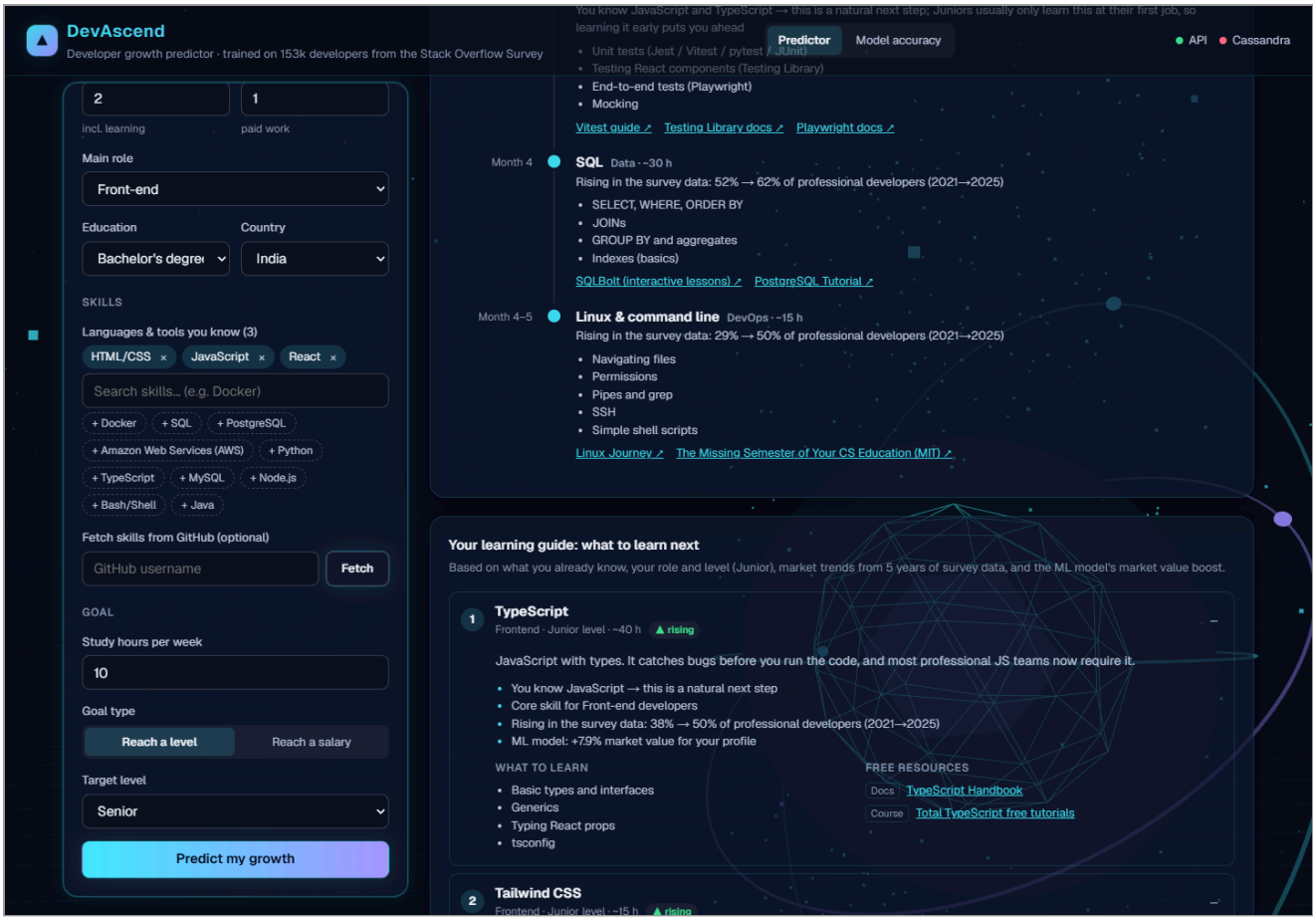


Figure 5: Learning guide with reasons for each recommendation

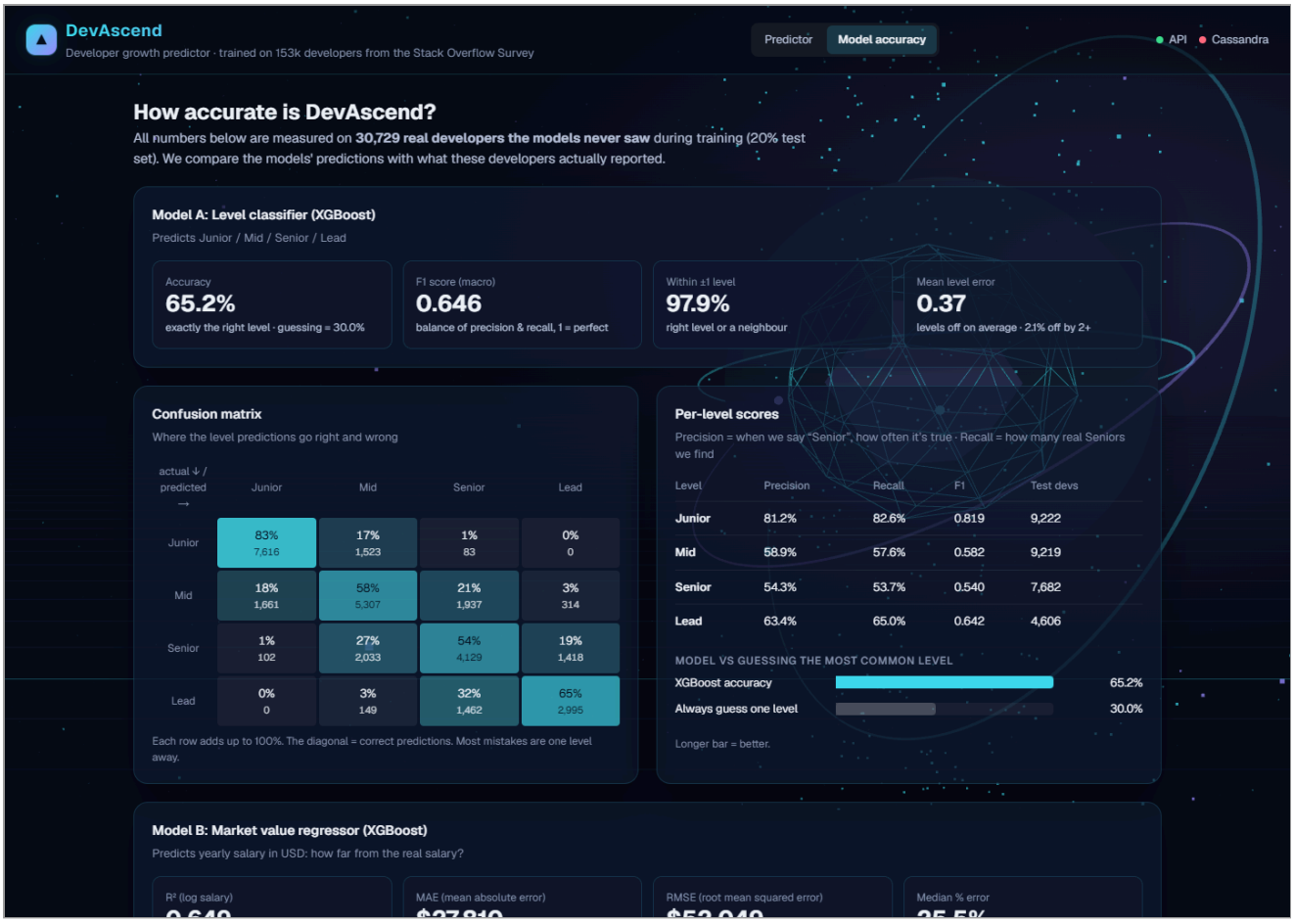


Figure 6: Model accuracy page: metrics, confusion matrix and per-level scores

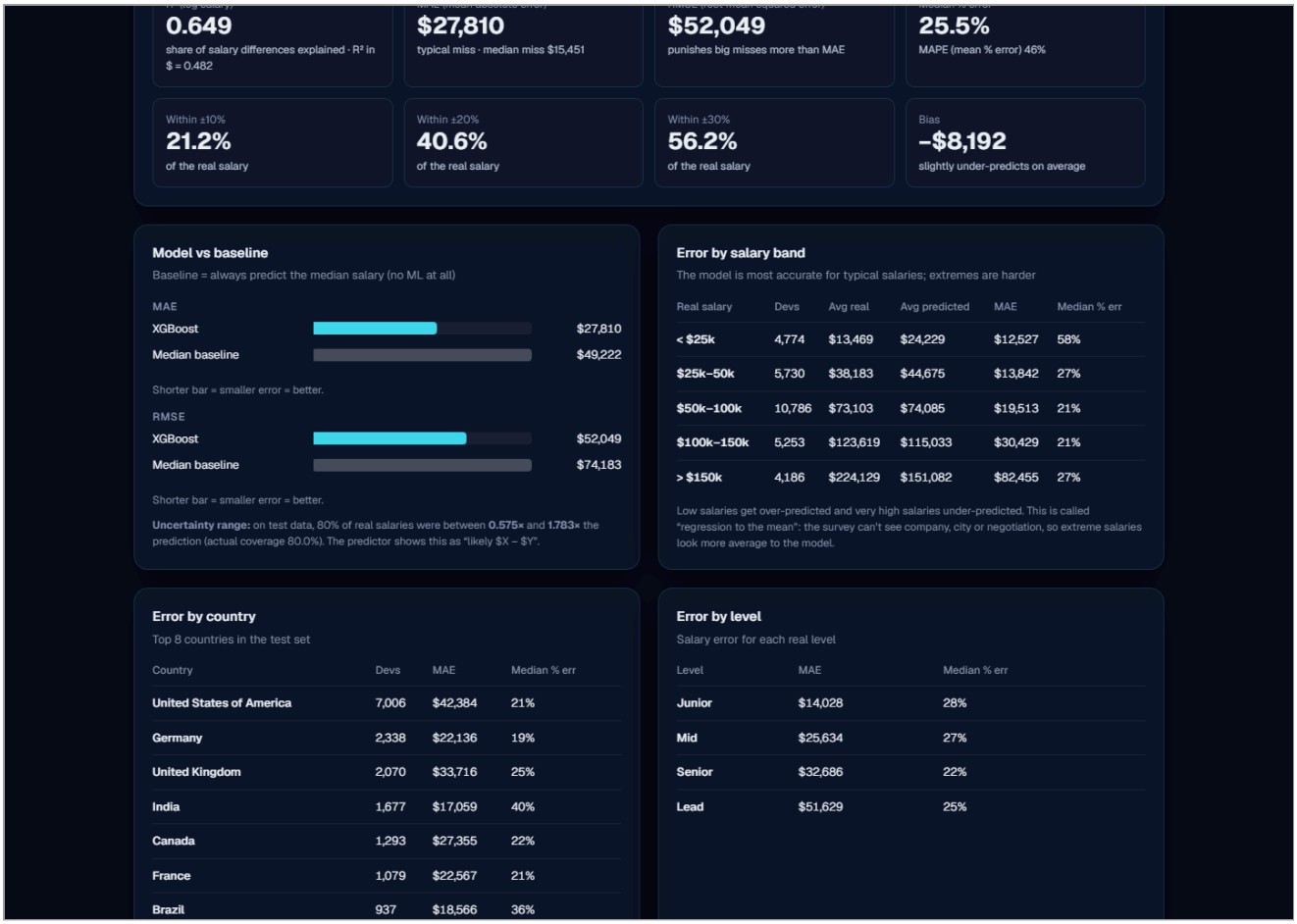
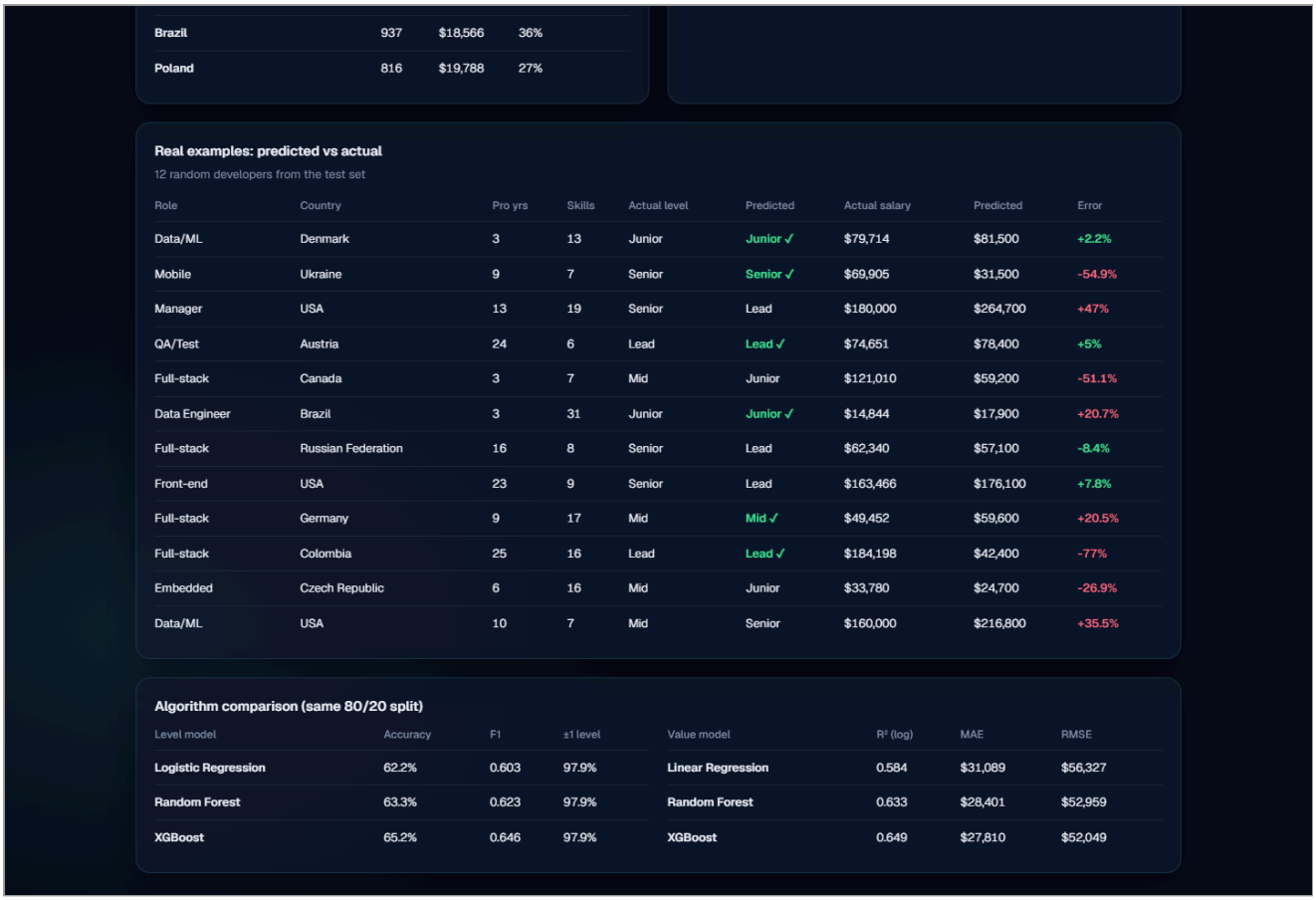


Figure 7: Market value metrics, model vs baseline and error by salary band



Brazil	937	$18,566	36%
Poland	816	$19,788	27%

Real examples: predicted vs actual

12 random developers from the test set

Role	Country	Pro yrs	Skills	Actual level	Predicted	Actual salary	Predicted	Error
Data/ML	Denmark	3	13	Junior	Junior ✓	$79,714	$81,500	+2.2%
Mobile	Ukraine	9	7	Senior	Senior ✓	$69,905	$31,500	-54.9%
Manager	USA	13	19	Senior	Lead	$180,000	$264,700	+47%
QA/Test	Austria	24	6	Lead	Lead ✓	$74,651	$78,400	+5%
Full-stack	Canada	3	7	Mid	Junior	$121,010	$59,200	-51.1%
Data Engineer	Brazil	3	31	Junior	Junior ✓	$14,844	$17,900	+20.7%
Full-stack	Russian Federation	16	8	Senior	Lead	$62,340	$57,100	-8.4%
Front-end	USA	23	9	Senior	Lead	$163,466	$176,100	+7.8%
Full-stack	Germany	9	17	Mid	Mid ✓	$49,452	$59,600	+20.5%
Full-stack	Colombia	25	16	Lead	Lead ✓	$184,198	$42,400	-77%
Embedded	Czech Republic	6	16	Mid	Junior	$33,780	$24,700	-26.9%
Data/ML	USA	10	7	Mid	Senior	$160,000	$216,800	+35.5%

Algorithm comparison (same 80/20 split)

Level model	Accuracy	F1	±1 level	Value model	R² (log)	MAE	RMSE
Logistic Regression	62.2%	0.603	97.9%	Linear Regression	0.584	$31,089	$56,327
Random Forest	63.3%	0.623	97.9%	Random Forest	0.633	$28,401	$52,959
XGBoost	65.2%	0.646	97.9%	XGBoost	0.649	$27,810	$52,049

Figure 8: Error by country and level, and real test developers with predicted vs actual values

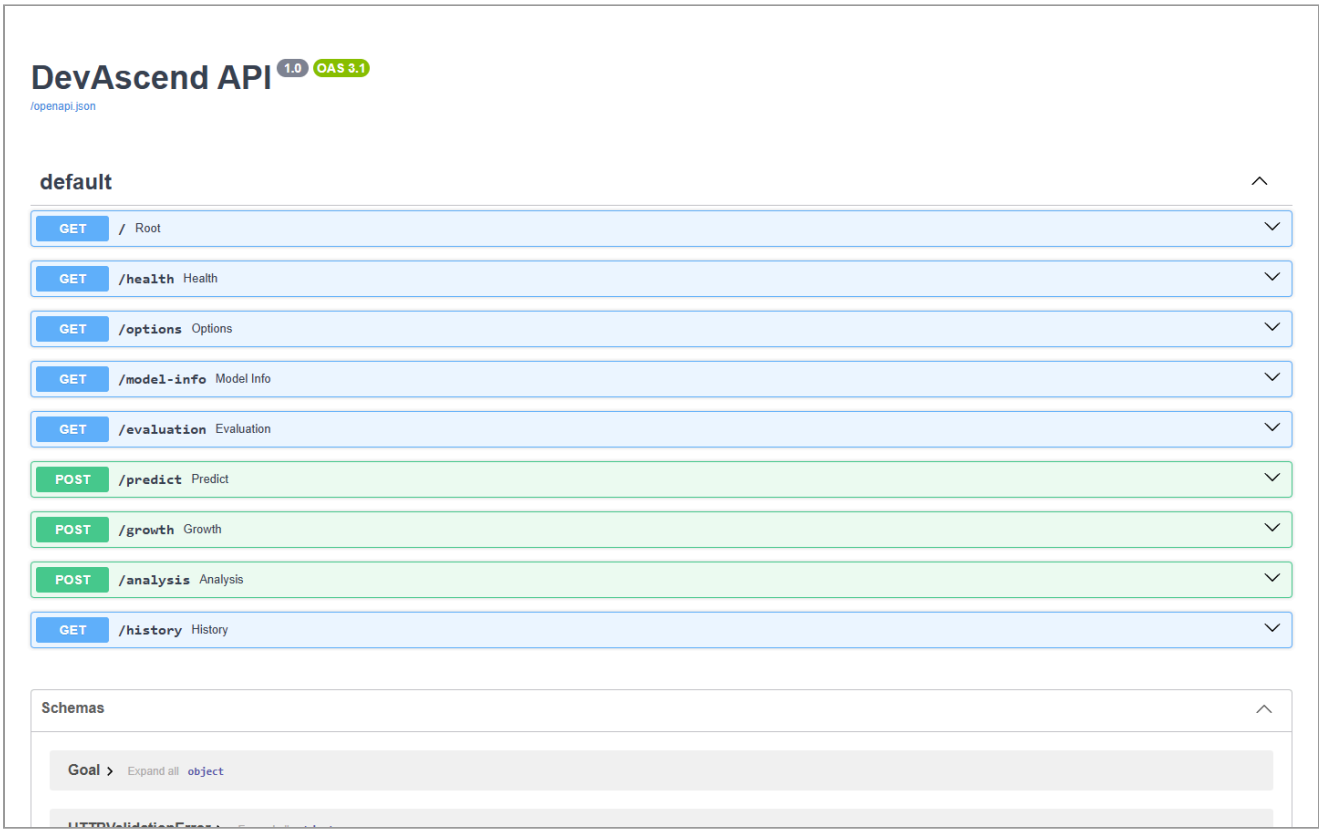


Figure 9: FastAPI interactive documentation (/docs) with all endpoints

5.14 Data analysis and model results

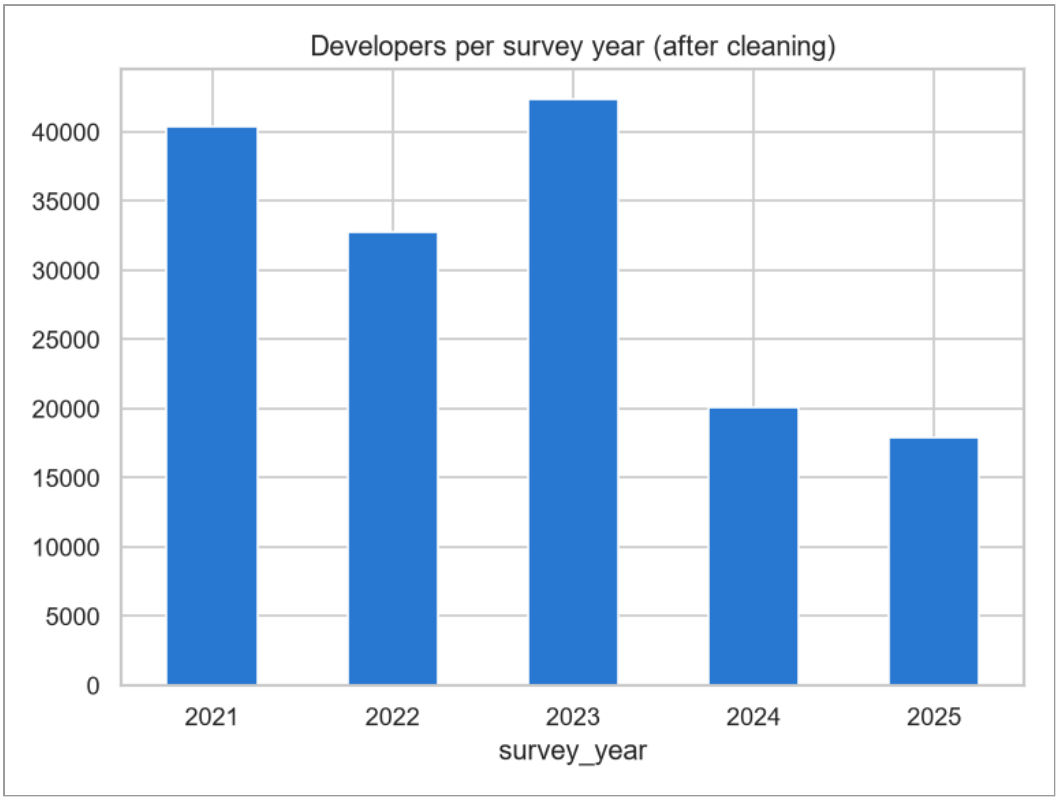


Figure 10: Survey responses per year stored in Cassandra

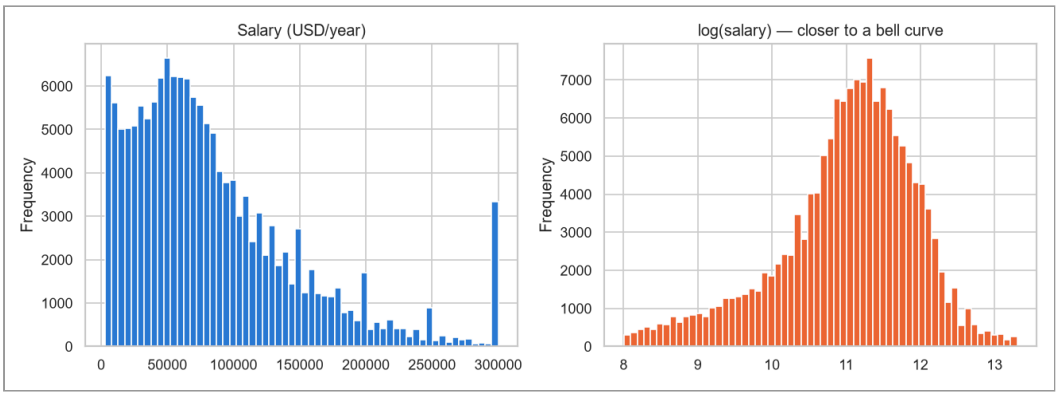


Figure 11: Salary distribution (right-skewed, so the model learns log salary)



Figure 12: Experience vs salary

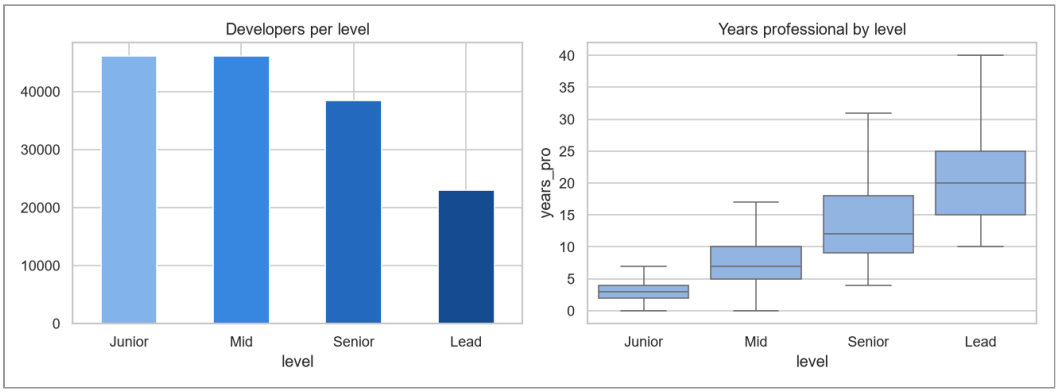


Figure 13: Level label distribution

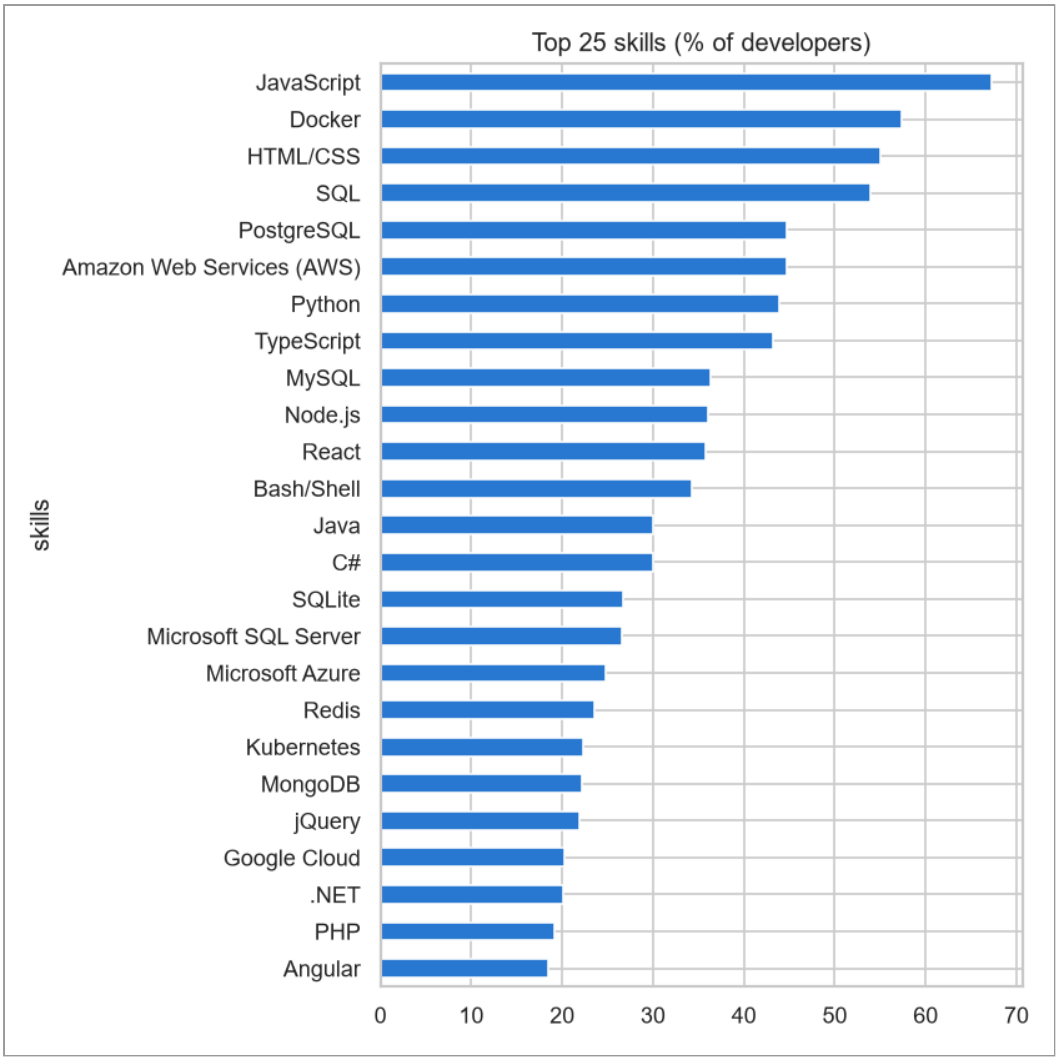


Figure 14: Most common skills

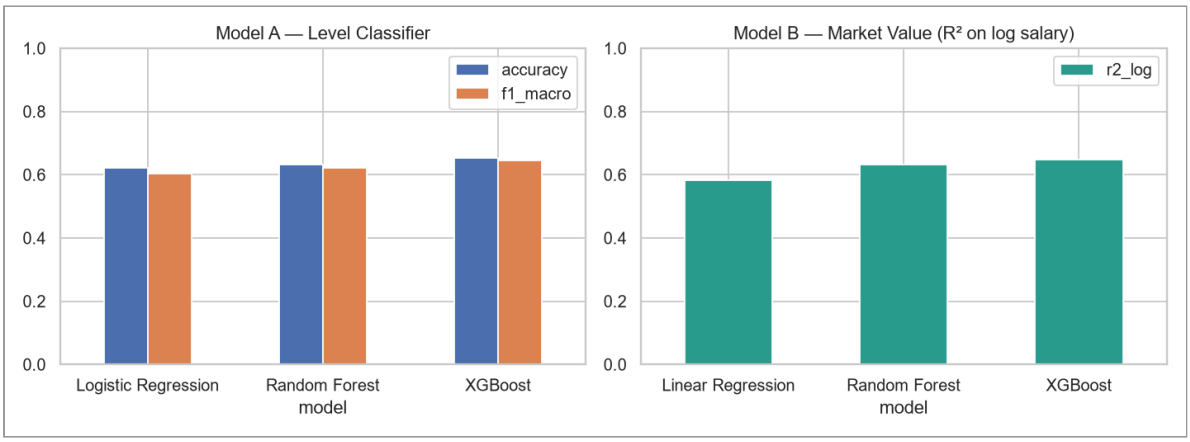


Figure 15: Comparison of the three algorithms for both tasks

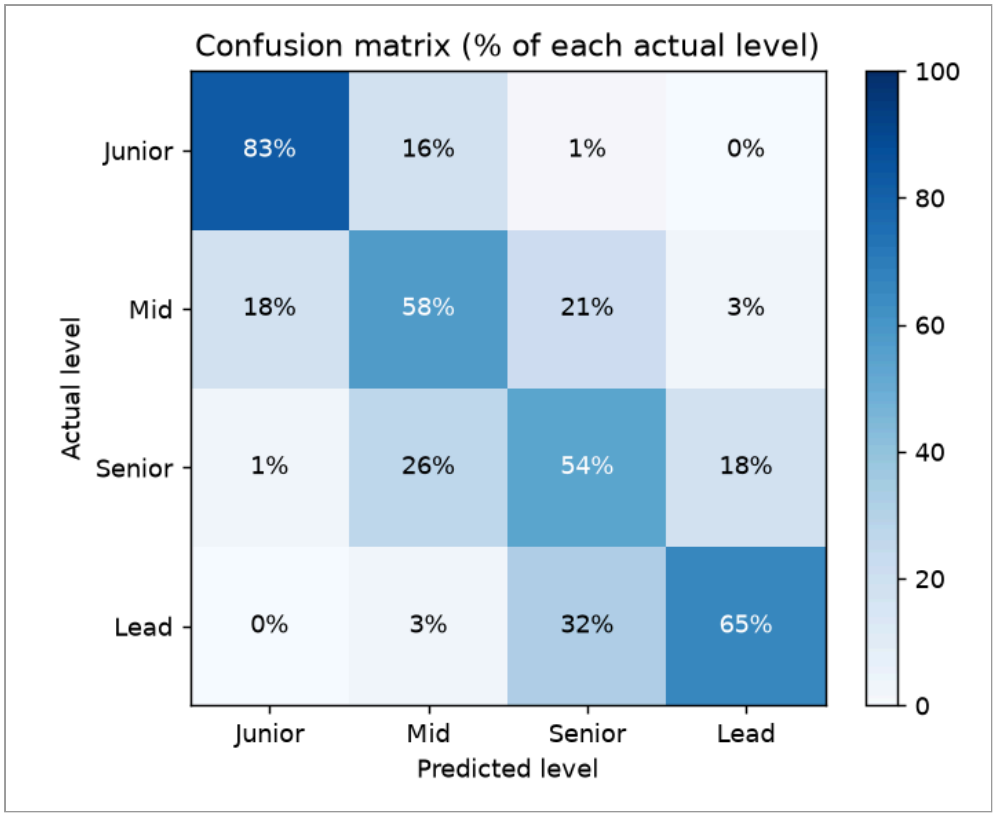


Figure 16: Confusion matrix of the XGBoost level classifier (% of each actual level)

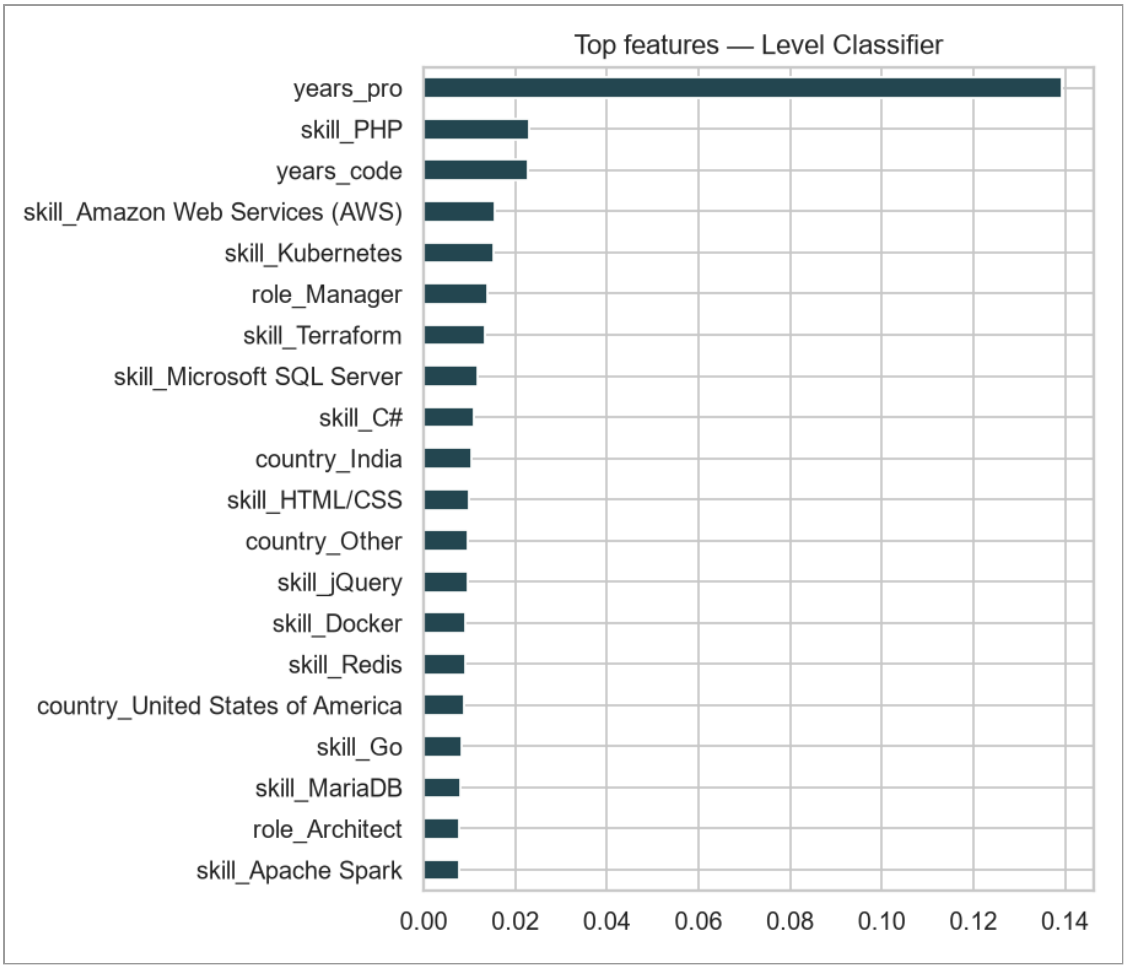


Figure 17: Feature importance, level model

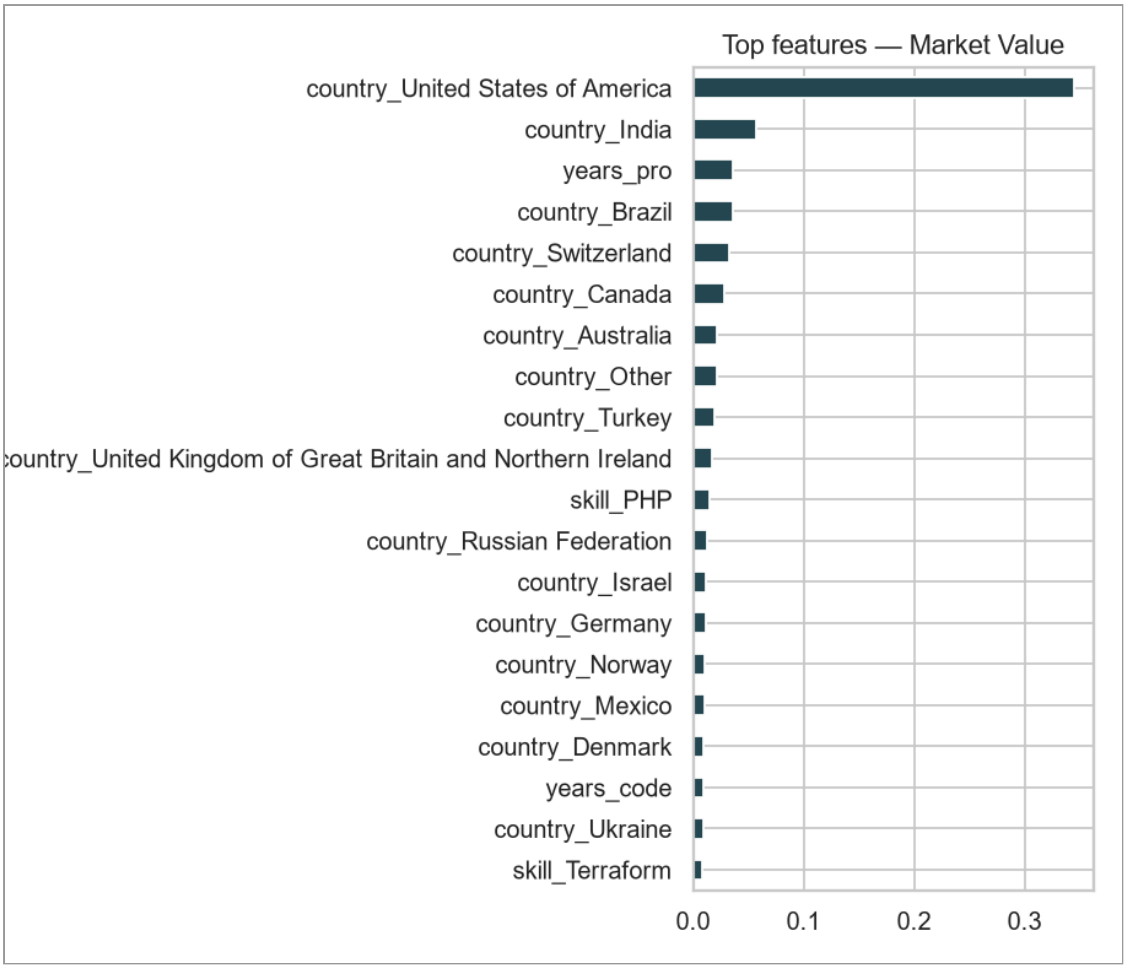


Figure 18: Feature importance, market value model

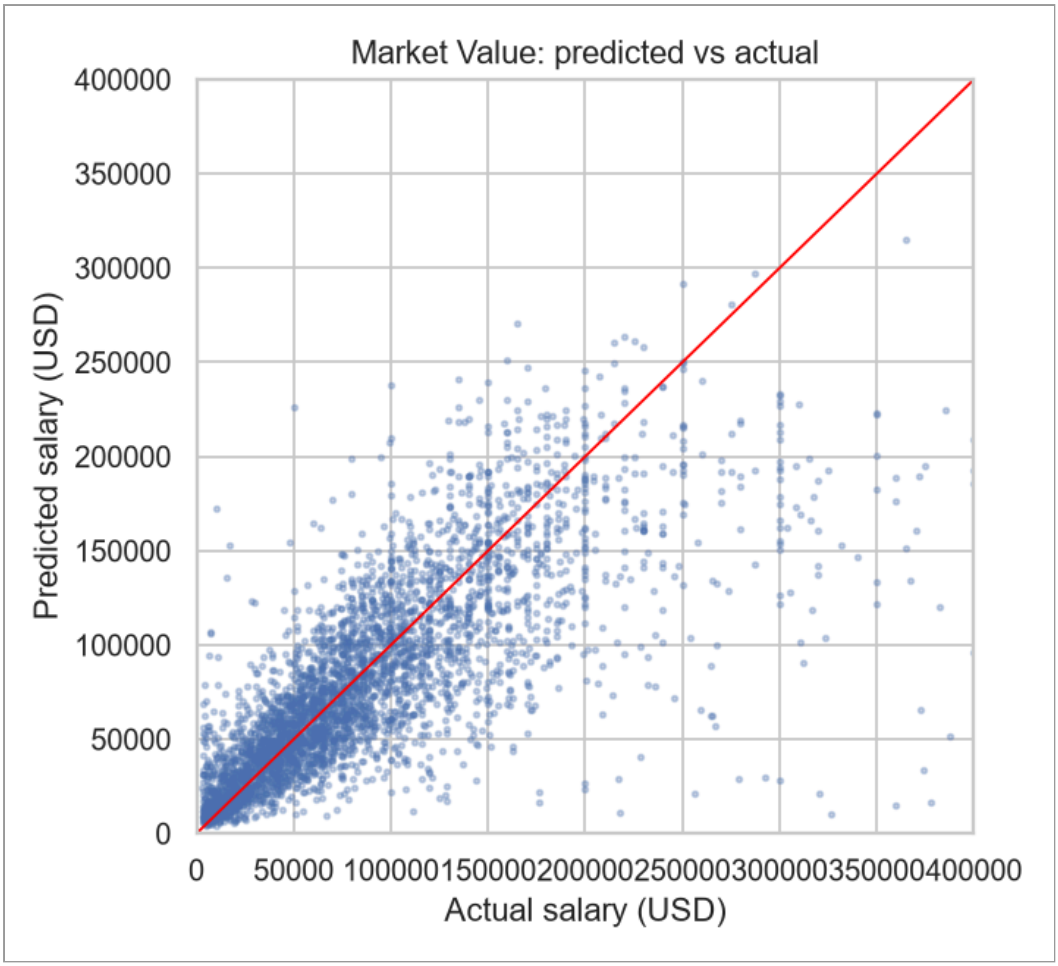


Figure 19: Predicted vs actual salary on the test set

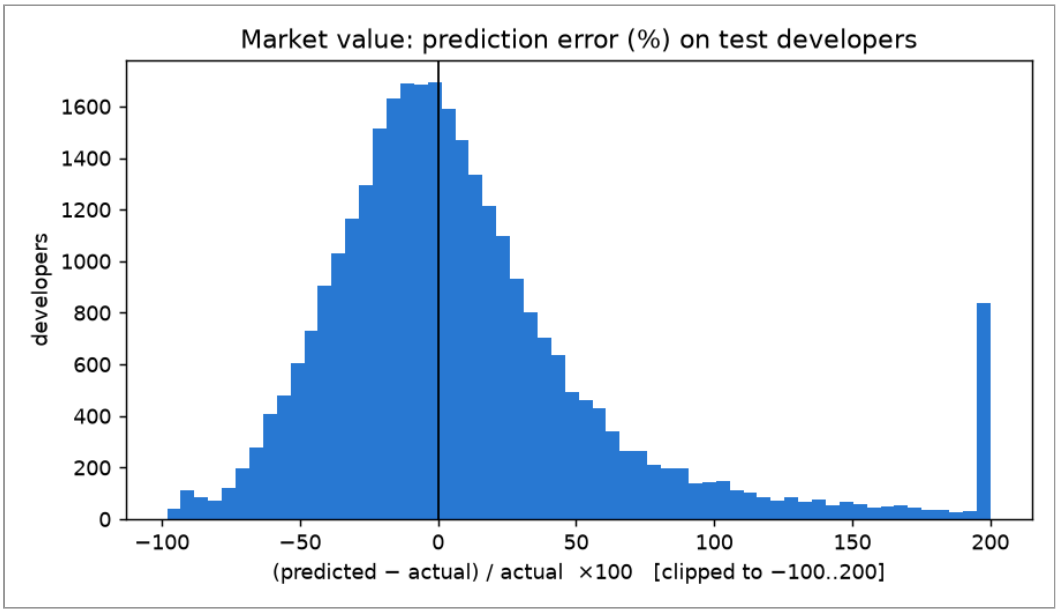


Figure 20: Distribution of prediction errors

6. CONCLUSION

6.1 Results

Model A: level	Accuracy	F1 (macro)	Within ±1 level
Logistic Regression	62.2%	0.603	97.9%
Random Forest	63.3%	0.623	97.9%
XGBoost (selected)	**65.2%**	**0.646**	**97.9%**

Model B: market value	R^2 (log)	MAE	RMSE
Linear Regression	0.584	$31,089	$56,327
Random Forest	0.633	$28,401	$52,959
XGBoost (selected)	**0.649**	**$27,810**	**$52,049**
Baseline (median salary)	0	$49,222	$74,183

XGBoost wins both tasks because boosting captures non-linear effects and interactions (for example, Kubernetes is worth more in some countries and roles) and handles many sparse 0/1 skill columns well. Almost every level mistake is one level away, which is expected because levels are cut points on a continuous score. Salary predictions have a 25.5% median error, and the 80% range ("likely $X – $Y") covered exactly 80.0% of test developers.

6.2 Conclusion

DevAscend shows a complete data-science system: data stored and served from Apache Cassandra, careful cleaning, a fairly defined label, a model comparison on a shared split, honest evaluation against baselines and real values, and a simulation that turns static predictions into a growth plan. The hybrid recommender fills the gap that the data alone cannot cover, giving junior developers concrete next steps with free resources, all delivered through a FastAPI backend and a 3D Next.js web application.

6.3 Limitations

- **Simulated future:** the data is cross-sectional, so the same person is never observed over years.
- **Correlation, not causation:** e.g. Kafka users may earn more partly because Kafka is used at large companies.
- **Salary as a proxy:** pay also depends on company, city and negotiation, which the survey does not record.

- **Estimates:** study hours per topic come from the knowledge base, not from data.
- **Survey bias:** respondents are not a perfect sample of all developers.

6.4 Future scope

- Collect data that follows the same users over time.
- Add city and company size as features.
- Add SHAP explanations and tune hyper-parameters with cross-validation.
- Use quantile regression for better ranges.
- Run Cassandra as a three-node cluster with replication factor 3.

6.5 References

1. Stack Overflow Developer Survey 2021–2025, survey.stackoverflow.co.
2. Apache Cassandra documentation.
3. scikit-learn user guide; XGBoost documentation.
4. FastAPI, Next.js and Three.js documentation.
5. Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
6. Breiman, L. (2001). Random Forests. Machine Learning 45, 5–32.